

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(УНИВЕРСИТЕТ ИТМО)

Факультет «Систем управления и робототехники»

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

По дисциплине «Частотные методы»
на тему: «Жесткая фильтрация»

Студент:
Охрименко Ева

Преподаватели:
Догадин Егор Витальевич
Пашенко Артем Витальевич

г. Санкт-Петербург
2025

Содержание

1	Task. Жесткие фильтры	2
1.1	Краткое условие	2
1.2	Убираем высокие частоты	2
1.2.1	Предподготовка	2
1.2.2	Фиксирую b	3
1.2.3	Вывод	7
1.2.4	Фиксирую ν_0	7
1.2.5	Вывод	10
1.3	Убираем специфические частоты	10
1.3.1	Предподготовка	10
1.3.2	Исследование	11
1.3.3	Вывод	19
1.4	Убираем низкие частоты?	20
1.4.1	Предподготовка	20
1.4.2	Графики	20
1.4.3	Выводы	22
2	Task. Фильтрация звука	22
2.1	Краткое условие	22
2.2	График исходного сигнала	23
3	Выводы	24
4	Приложение	24

1 Task. Жесткие фильтры

1.1 Краткое условие

Рассмотрите функцию $g(t)$, заданную как:

$$g(t) = \begin{cases} a, & t \in [t_1, t_2], \\ 0, & t \notin [t_1, t_2], \end{cases}$$

и её зашумлённую версию:

$$u(t) = g(t) + b\xi(t) + c \sin(dt),$$

где $\xi(t) \sim U[-1, 1]$ — белый шум, а b, c, d — параметры.

- При $c = 0$ найдите Фурье-образ $u(t)$, обнулите его вне $[-\nu_0, \nu_0]$ и выполните обратное преобразование. Исследуйте влияние ν_0 и b .
- При ненулевых b, c, d обнулите Фурье-образ на выбранных частотах, подавляя шум и гармонику. Исследуйте влияние параметров.
- бнулите Фурье-образ в окрестности $\nu = 0$, пропустите сигнал через фильтр и оцените результат.

Ожидаемые результаты:

Графики исходного, зашумлённого и фильтрованного сигналов, а также их Фурье-образов. Выводы по каждому пункту.

1.2 Убираем высокие частоты

1.2.1 Предподготовка

Для начала выберу все нужные параметры для этого задания:

$$a = 4, t_0 = 0, t_1 = 3, c = 0, d = 5, b = 0.5$$

Тогда у меня получится прямоугольная функция:

$$g(t) = \begin{cases} 4, & t \in [0, 3], \\ 0, & t \notin [0, 3], \end{cases}$$

Теперь посмотрим, какая функция белого шума получилась:

$$u(t) = g(t) + 0.5\xi(t) + 0 \sin(dt),$$

где $\xi(t) \sim U[-1, 1]$ — равномерное распределение на интервале $[-1, 1]$.

Слагаемое с синусом отсутствует, поэтому колебаний у шума также не будет.

Теперь посмотрим на график функций $g(t)$ и $u(t)$:

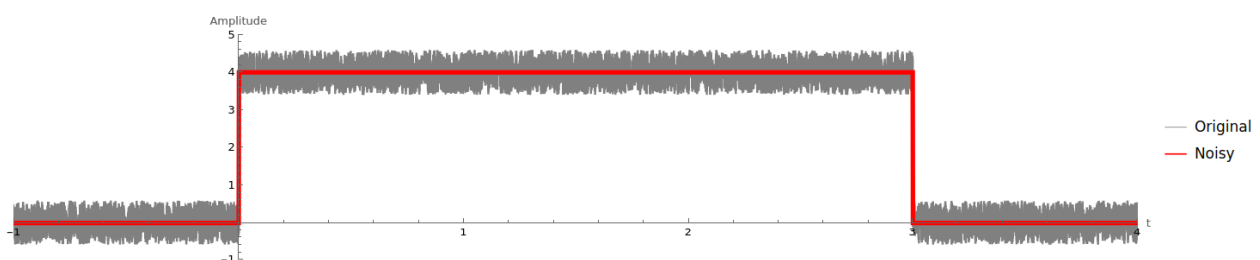


Рис. 1: График $g(t)$ и $u(t)$

Алгоритм фильтрации:

- Вычисляю спектр сигнала: `Ushifted`
- Создаю маску, обнуляющую частоты вне диапазона $[-\nu; \nu]$: `filterMask`
- Применяю фильтр: умножаю маску на спектр `Ufiltered = Ushifted*filterMask` и делаю обратное преобразование Фурье

1.2.2 Фиксирую b

Для начала выберу значения $\nu_0 = 0.5$ и $b = 0.5$. Теперь посмотрим на график получившегося отфильтрованного сигнала при этих значениях. Также я приведу графики модулей фурые образов сигнала $g(t)$, зашумленного сигнала $u(t)$ и отфильтрованного сигнала.

Здесь и далее стоит придерживаться такой легенды на графиках:

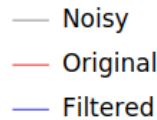


Рис. 2: Значения цветов

На графике фурые-образов также можно заметить две вертикальные синие пунктирные черты, между ними сигнал не обнуляется.

В этой части задания я зафиксирую параметр $b = 0.5$ и буду исследовать влияние на поведение функций параметра ν_0 . Выберу несколько $\nu_0 = \{0.5, 1, 1.5, 3, 5, 10\}$ и отрисую графики:

- Значения: $\nu_0 = 0.5$ и $b = 0.5$

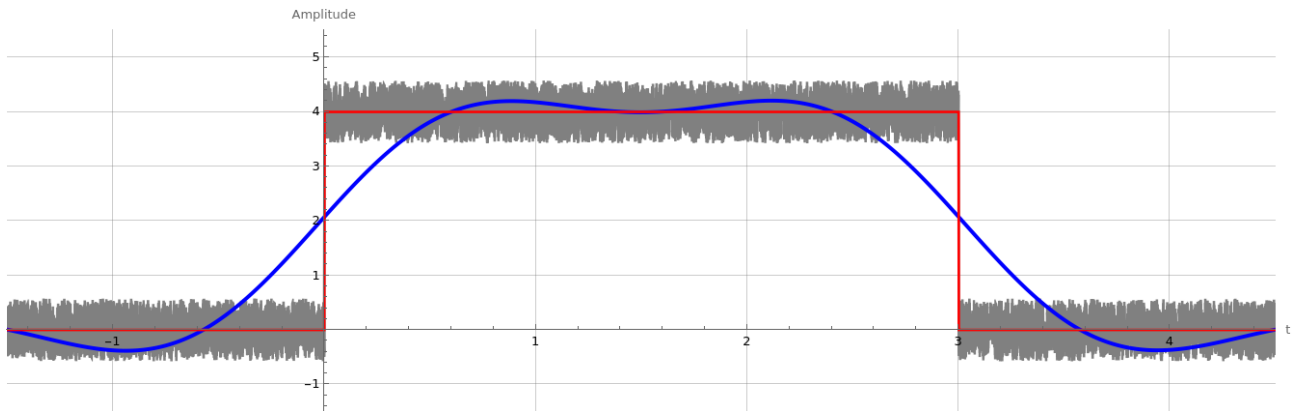


Рис. 3: Сигналы

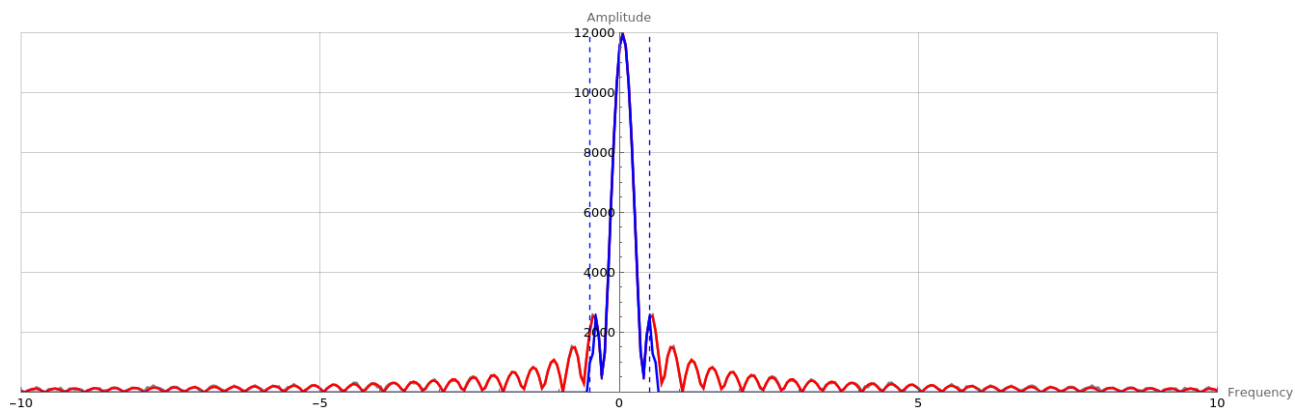


Рис. 4: Фурье-образы¹

- Значения: $\nu_0 = 1$ и $b = 0.5$

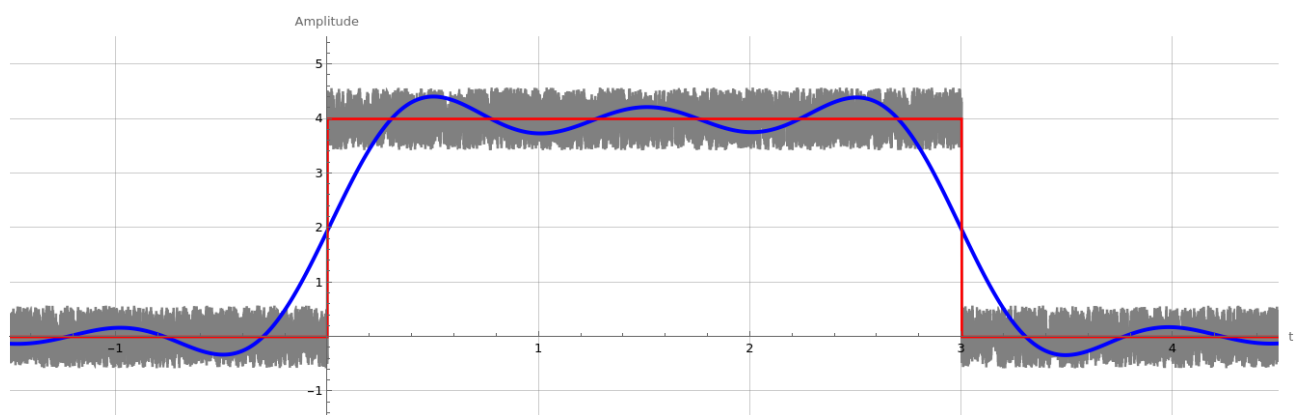


Рис. 5: Сигналы

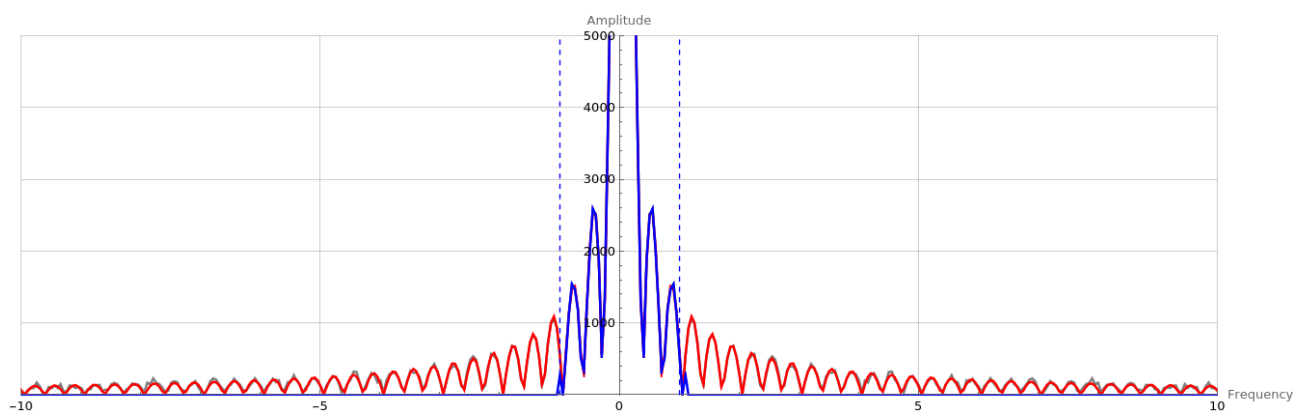


Рис. 6: Фурье-образы

- Значения: $\nu_0 = 1.5$ и $b = 0.5$

¹Далее в задании 1.2 я обрежу ось **Amplitude** до значения 5000, поскольку если не обрезать, основная часть будет не такой ясной для восприятия.

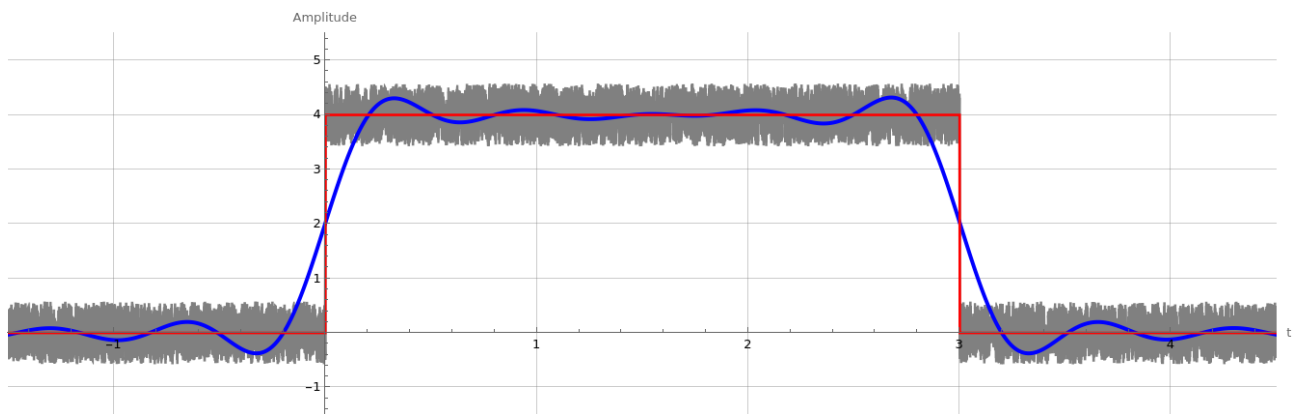


Рис. 7: Сигналы

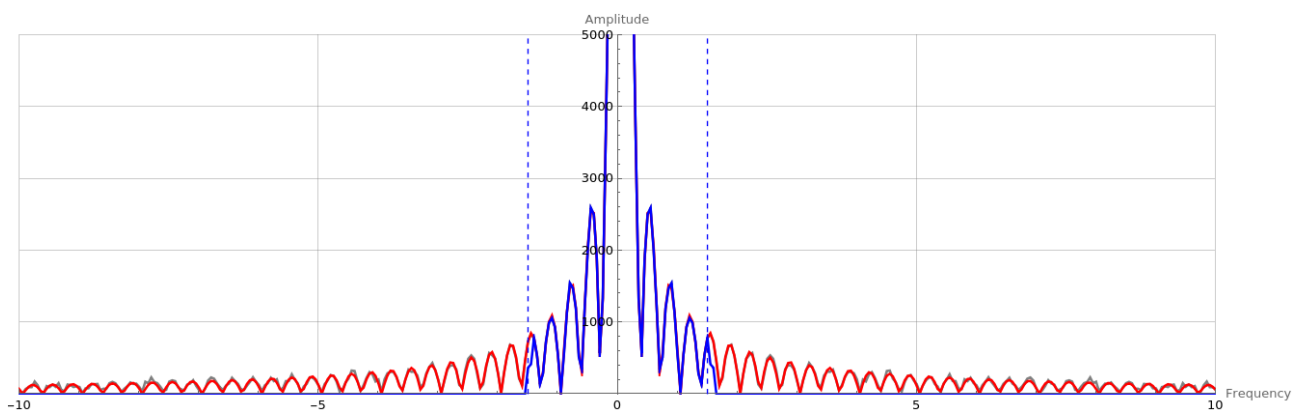


Рис. 8: Фурье-образы

- Значения: $\nu_0 = 3$ и $b = 0.5$

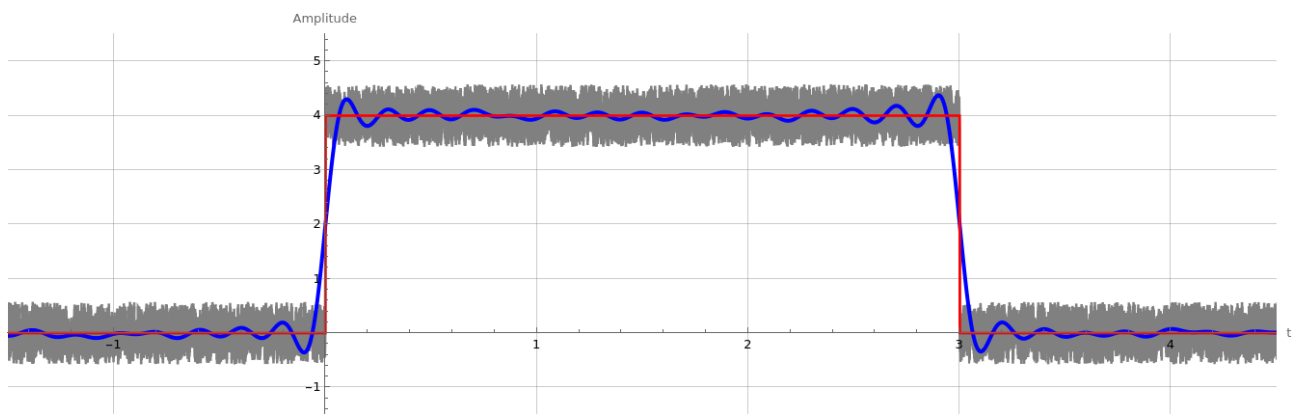


Рис. 9: Сигналы

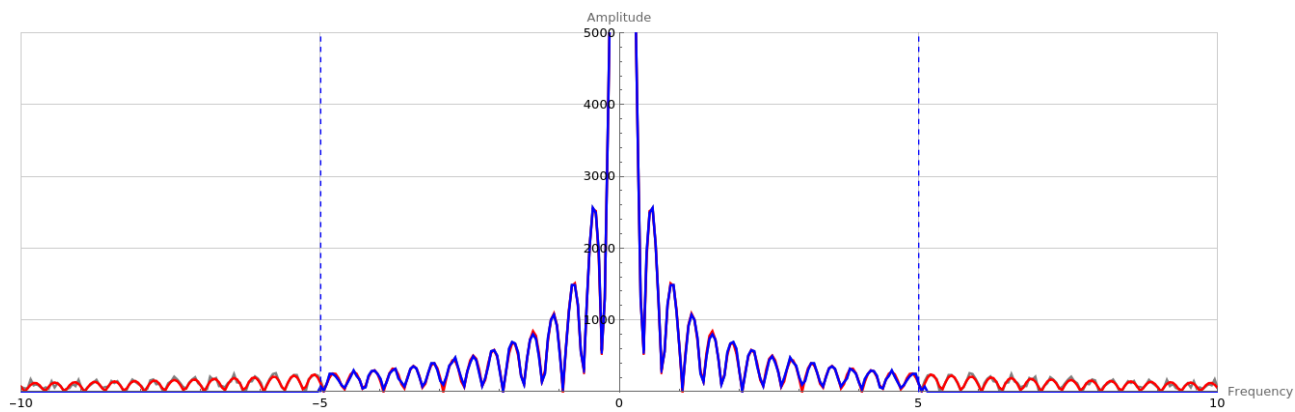


Рис. 10: Фурье-образы

- Значения: $\nu_0 = 5$ и $b = 0.5$

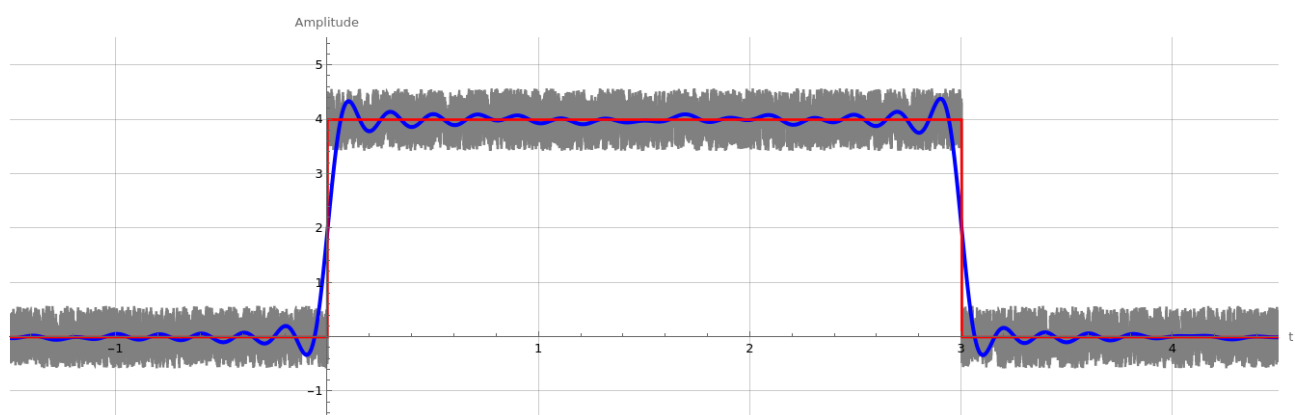


Рис. 11: Сигналы

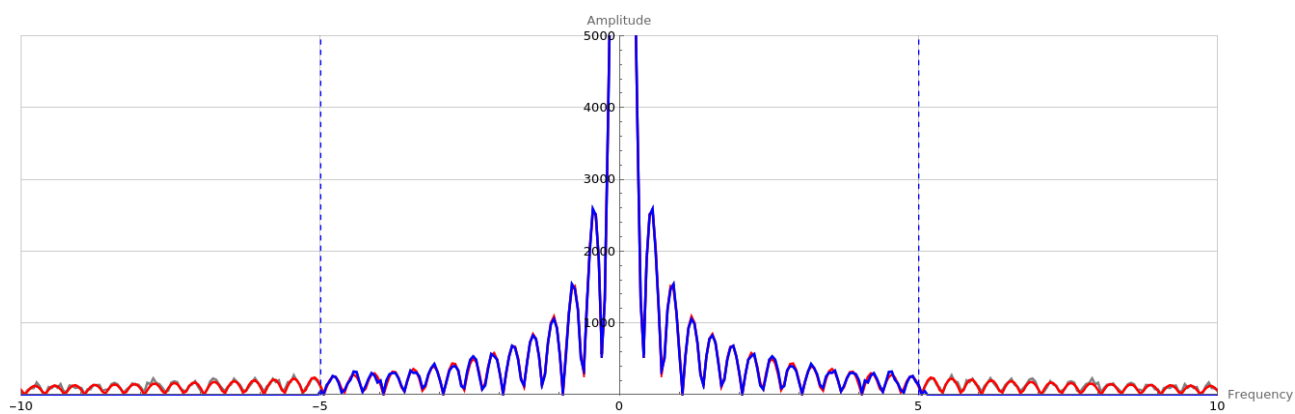


Рис. 12: Фурье-образы

- Значения: $\nu_0 = 10$ и $b = 0.5$

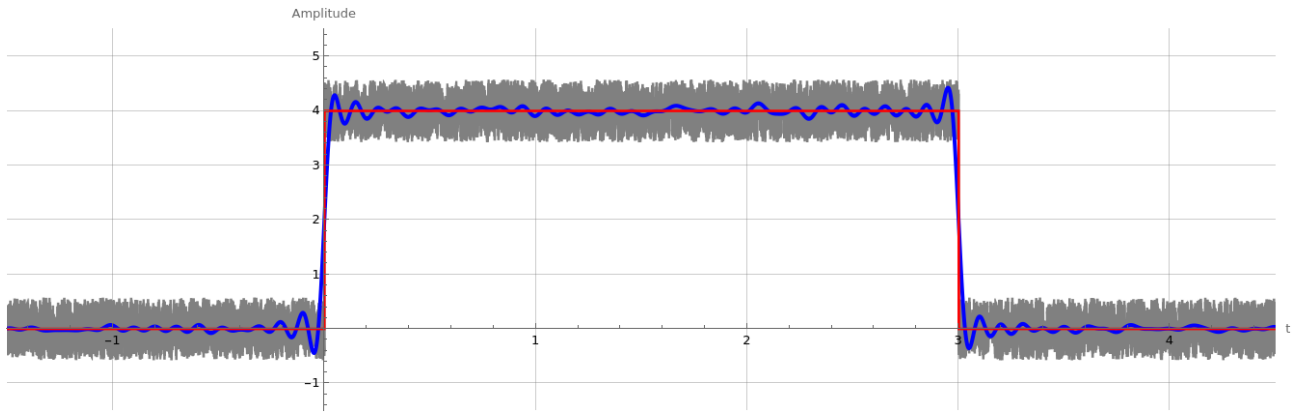


Рис. 13: Сигналы

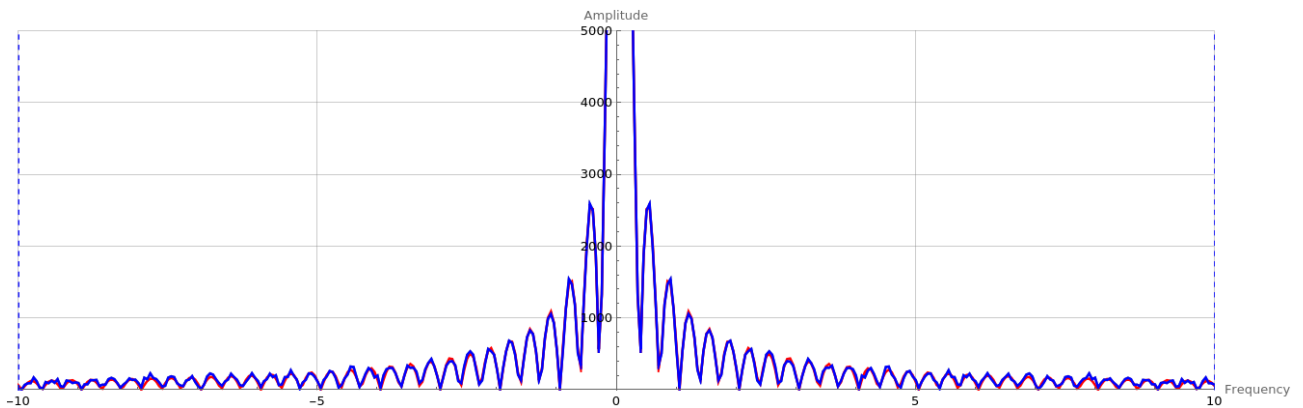


Рис. 14: Фурье-образы

1.2.3 Вывод

- С увеличением ν_0 изменялось количество колебаний отфильтрованного сигнала. Число гармоник увеличивалось, однако при большем значении ν_0 не всегда удавалось получить хорошо отфильтрованный сигнал. Наиболее удачным оказался график при параметрах $\nu_0 = 5$ и $b = 0.5$. На этом графике форма отфильтрованного сигнала практически идеально совпадает с исходной, а шум удалён наиболее эффективно.
- Можно заметить, что графики модулей Фурье-образов отфильтрованного сигнала и шума совпадают при всех выбранных значениях ν_0 . Теперь обратим внимание на синюю линию — модуль Фурье-образа. С увеличением ν_0 синий график постепенно начинает совпадать с остальными, практически полностью повторяя их форму. Это означает, что при увеличении частоты среза ν_0 фильтр пропускает больше частот, что приводит к лучшему сохранению формы сигнала.

1.2.4 Фиксирую ν_0

Для этого задания выберу несколько значений $b = \{0, 0.5, 1, 2\}$, чтобы исследовать поведение графиков при фиксированном значении $\nu_0 = 5$. Это значение было выбрано, поскольку ранее мне показалось, что при этом значении сигнал хорошо фильтруется. Также приведу в порядок частотную сетку, оставлю по оси частот значения $[-5; 5]$. Ниже рассмотрим эти графики:

- Значения: $\nu_0 = 5$ и $b = 0.0$

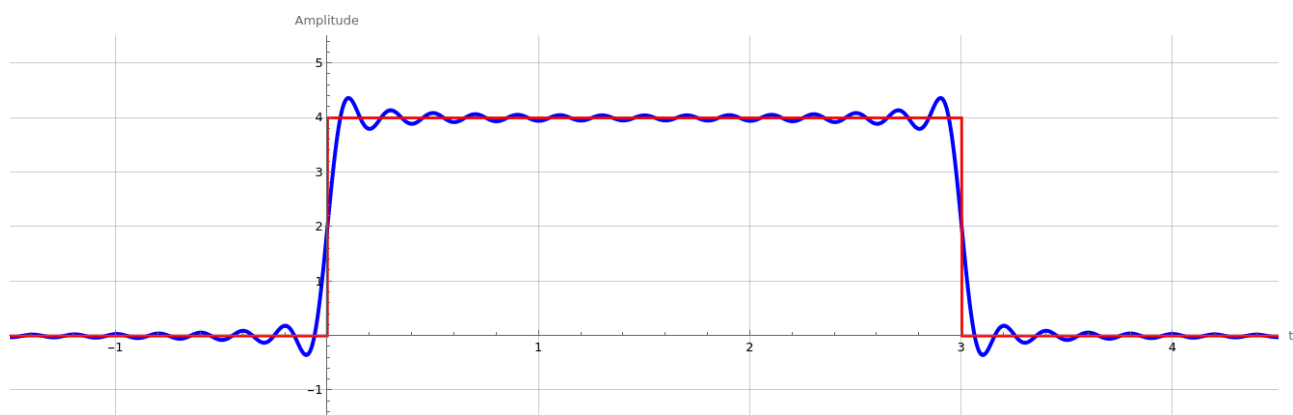


Рис. 15: Сигналы

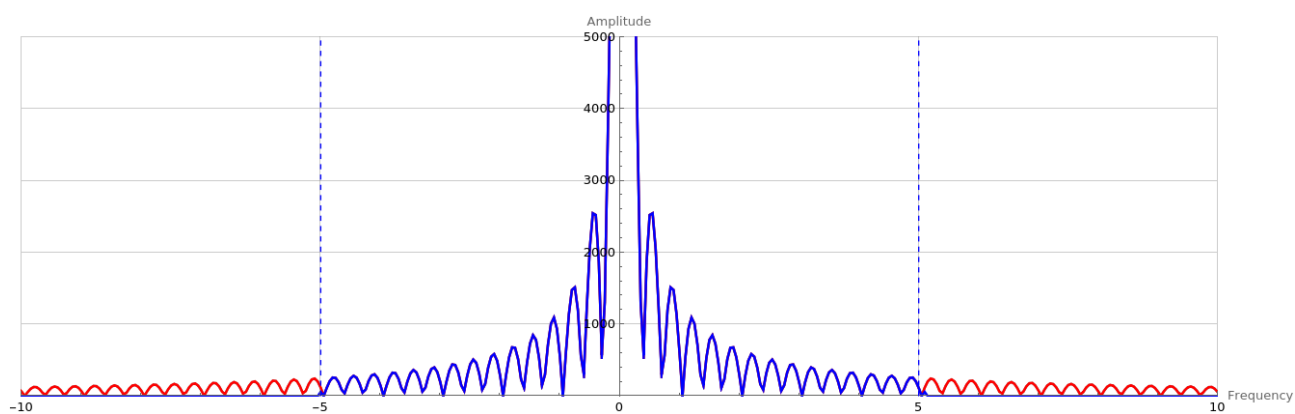


Рис. 16: Фурье-образы

- Значения: $\nu_0 = 5$ и $b = 0.5$

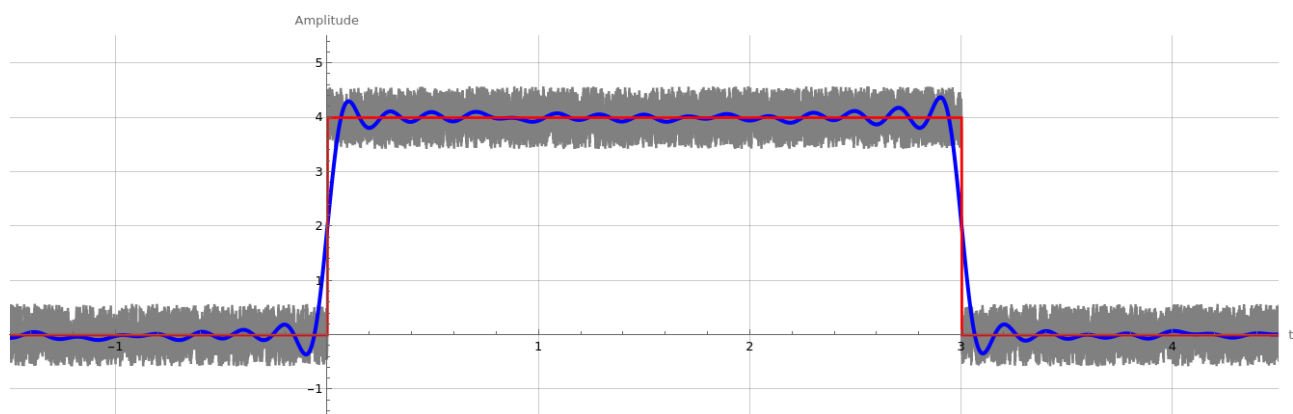


Рис. 17: Сигналы

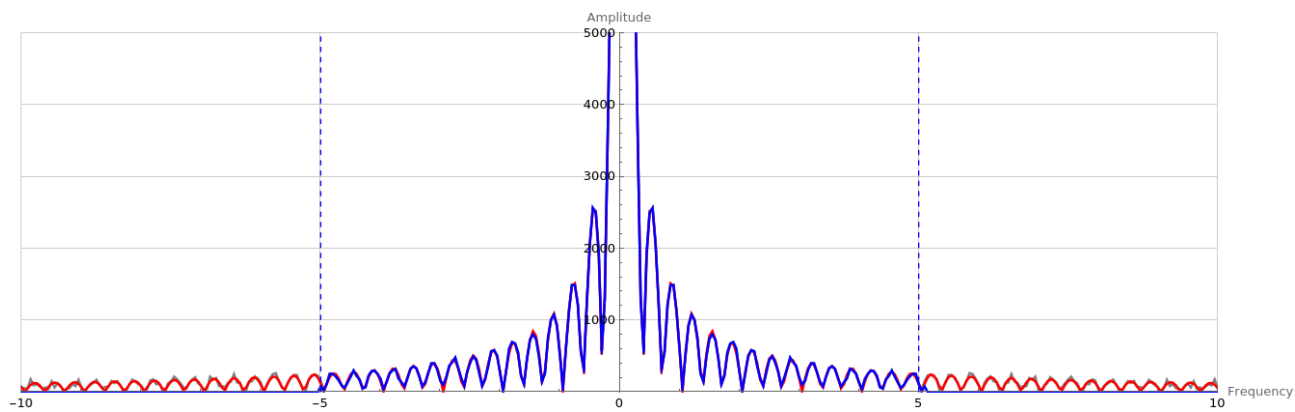


Рис. 18: Фурье-образы

- Значения: $\nu_0 = 5$ и $b = 1$

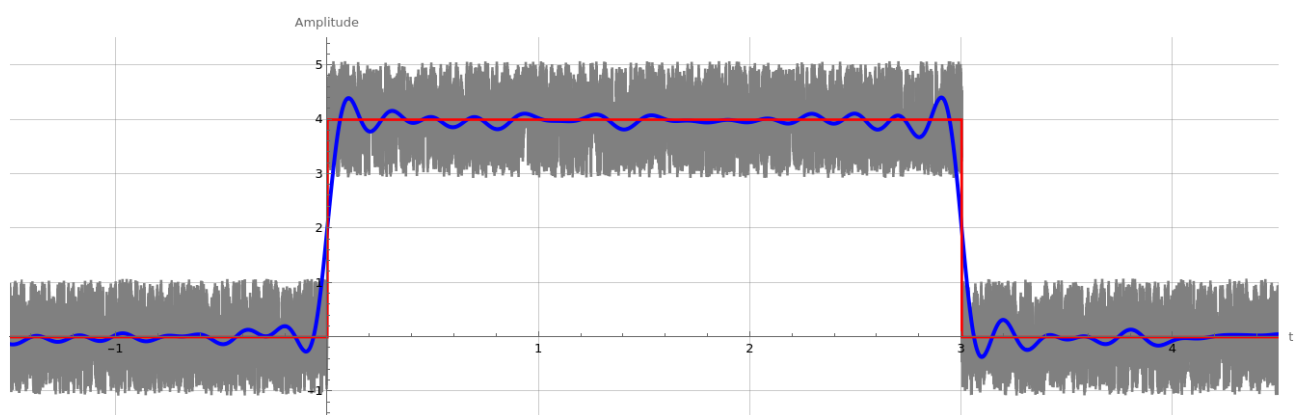


Рис. 19: Сигналы

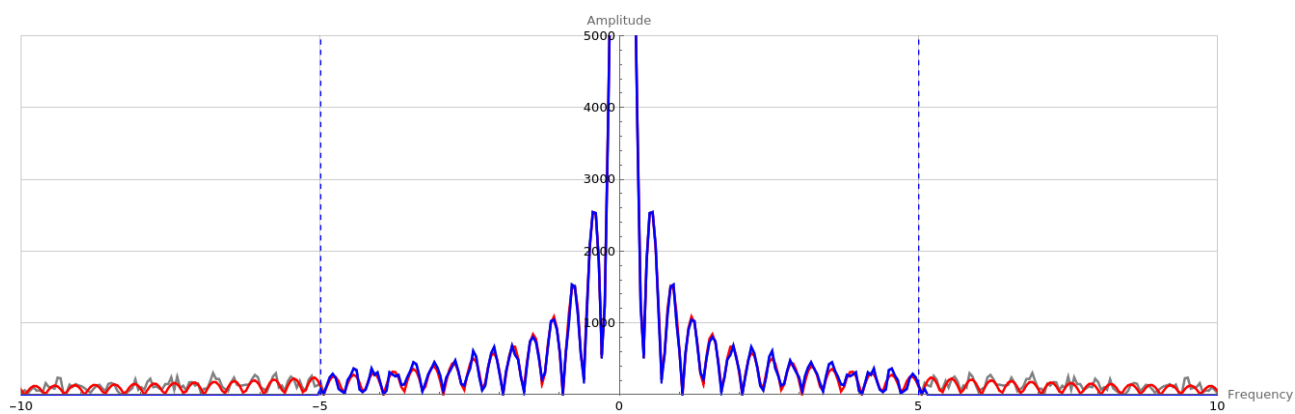


Рис. 20: Фурье-образы

- Значения: $\nu_0 = 5$ и $b = 2$

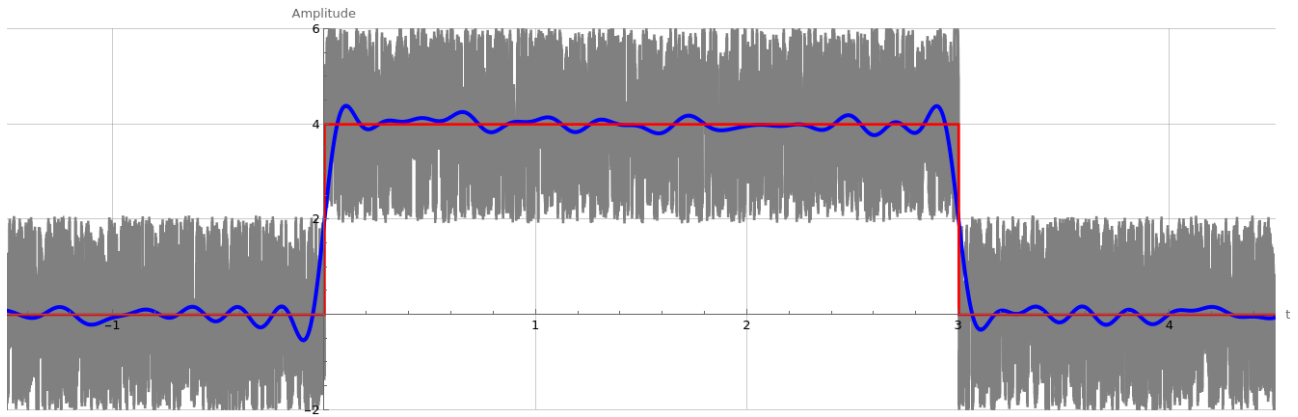


Рис. 21: Сигналы

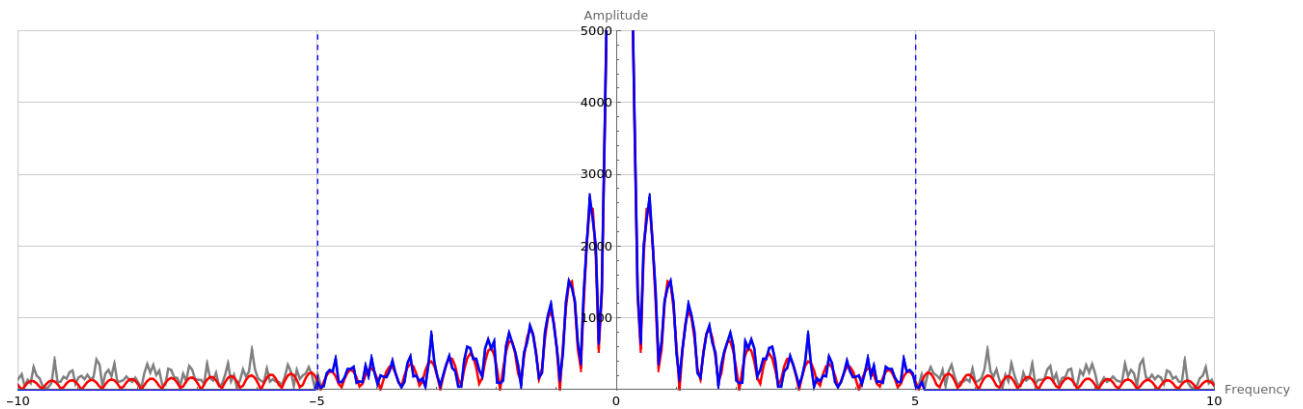


Рис. 22: Фурье-образы

1.2.5 Вывод

- При увеличении b увеличивается шум, а также растет амплитуда колебаний синего графика. То есть чем больше шум, тем больше помех в отфильтрованном сигнале.
- Спектр отфильтрованного сигнала близок к спектру $g(t)$, но остаются небольшие шумовые компоненты. Различия между модулями фурье-образов слабо заметны при моих параметрах b .

1.3 Убираем специфические частоты

1.3.1 Предподготовка

Для этого задания выберу параметры функций:

$$a = 4, t_0 = 0, t_1 = 3, c = 1, d = 15, b = 0.5$$

У меня получатся следующие функции:

$$g(t) = \begin{cases} 4, & t \in [0, 3], \\ 0, & t \notin [0, 3], \end{cases}$$

$$u(t) = g(t) + 0.5\xi(t) + \sin(2t),$$

где $\xi(t) \sim U[-1, 1]$ — равномерное распределение на интервале $[-1, 1]$.

Отрисую график для выбранных значений:

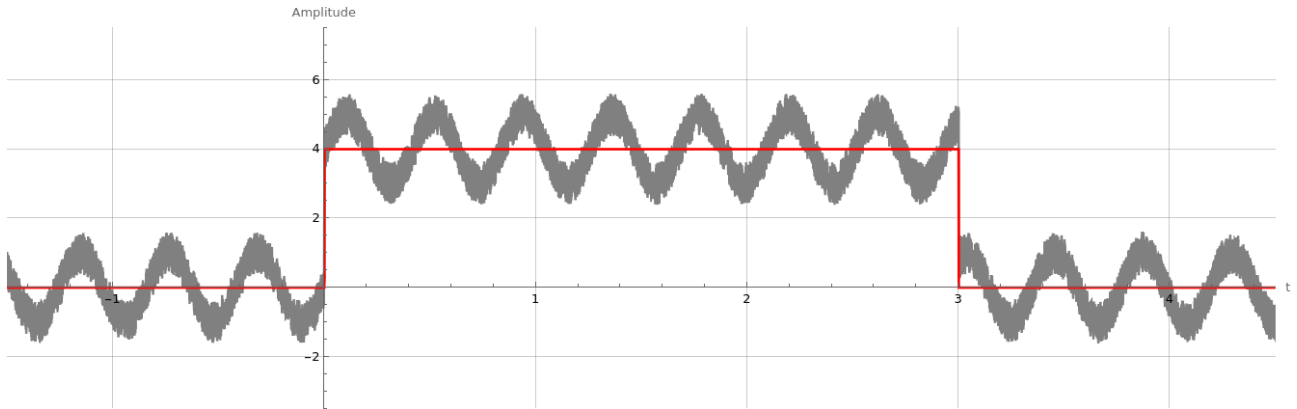


Рис. 23: Сигналы

В отличие от предыдущего пункта можно заметить, что шум пошел по синусоиде.

В этом задании нужно сделать совмещенный фильтр - он убирает не только низкие частоты, но и гармонические колебания. Отмечу, что в случае $b = 0$ убирать частоты не требуется, нужно только очистить сигнал от синусоиды.

Алгоритм фильтрации:

- Вычисляю спектр сигнала: `Ushifted`
- Создаю маску, обнуляющую узкую полосу вокруг частоты помехи $\text{harmFreq} = \frac{d}{2\pi}$ - это режекторный фильтр: `notchMask`. Ширина режектора регулируется параметром `notchWidth`
- Создаю маску, которая обнуляет частоты вне диапазона $[-v, v]$ - фильтр низких частот: `lowPassMask`
- Перемножаю маски поэлементно - получаю совмещенную маску: `combinedMask = lowPassMask*notchMask`. В случае, когда $b = 0$ я принимаю `lowPassMask = [1, 1, \dots, 1]`. Так как шума нет, мне нужно убрать только гармонические помехи
- Применяю фильтр: умножаю маску на спектр `Ufiltered = Ushifted*combinedMask` и делаю обратное преобразование Фурье

1.3.2 Исследование

Возьму набор параметров: $\nu_0 = 7, c = 1, d = 15, b = 0.5$ и менять поочередно значения, чтобы посмотреть его влияние на графики.

- Влияние параметра b

Выбранные параметры: $\nu_0 = 7, c = 1, d = 15, \mathbf{b} = \mathbf{0}$

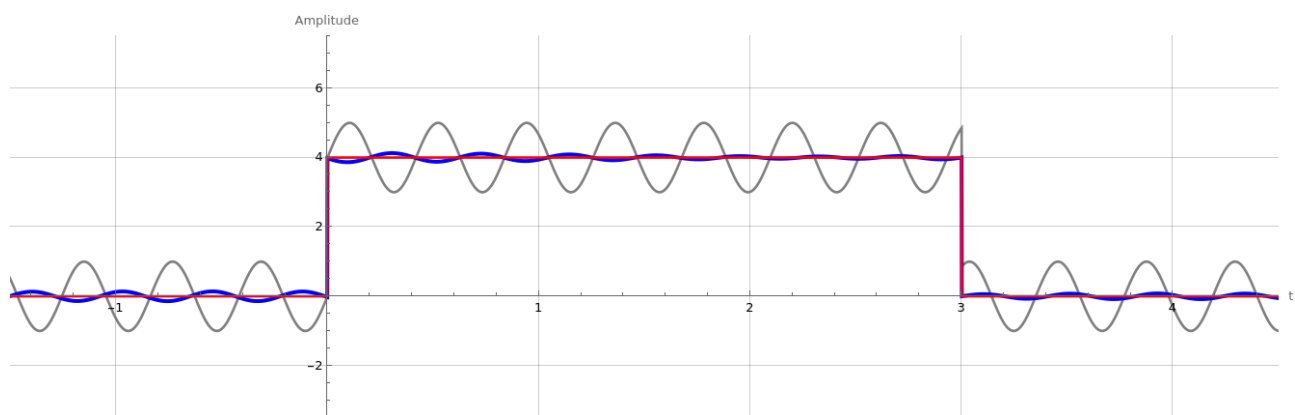


Рис. 24: Сигналы

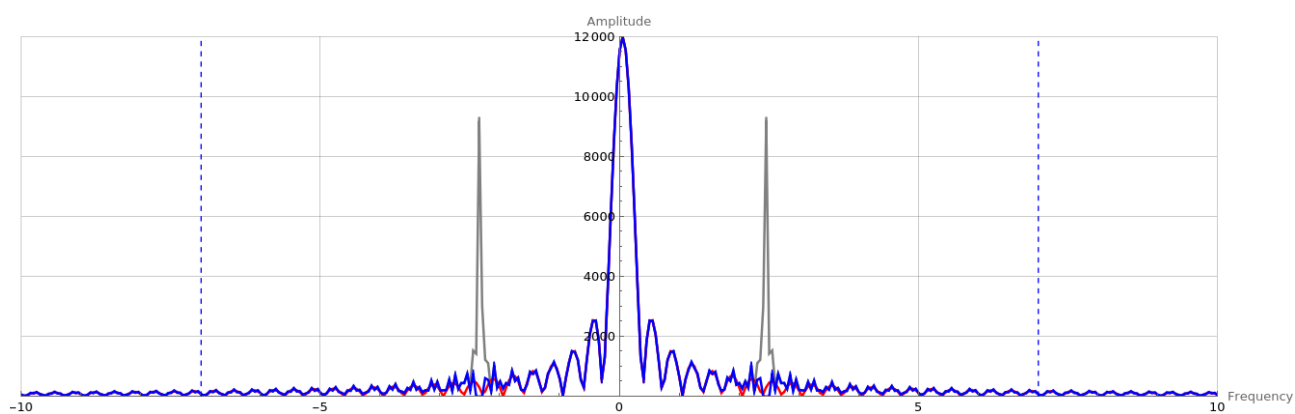


Рис. 25: Фурье-образы

Выбранные параметры: $\nu_0 = 7, c = 1, d = 15, \mathbf{b} = \mathbf{0.5}$

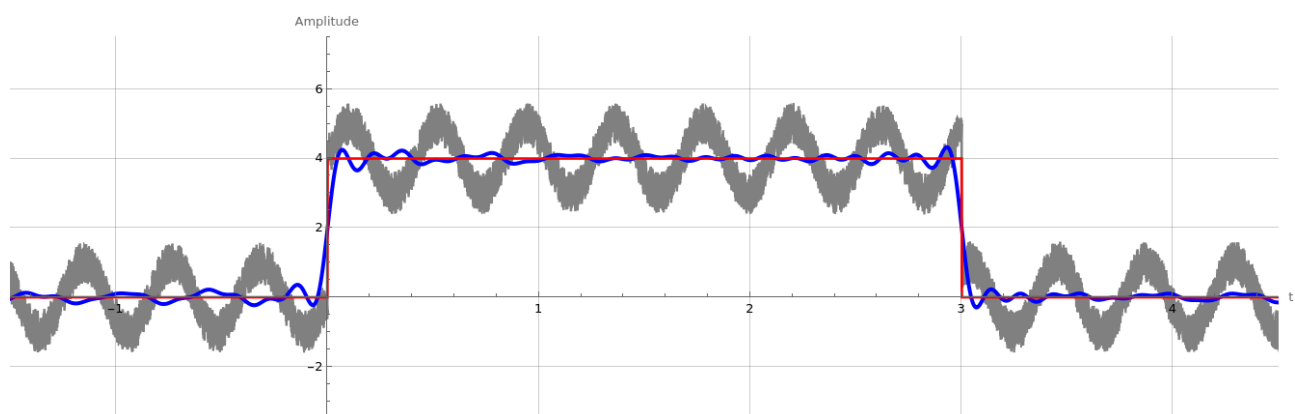


Рис. 26: Сигналы

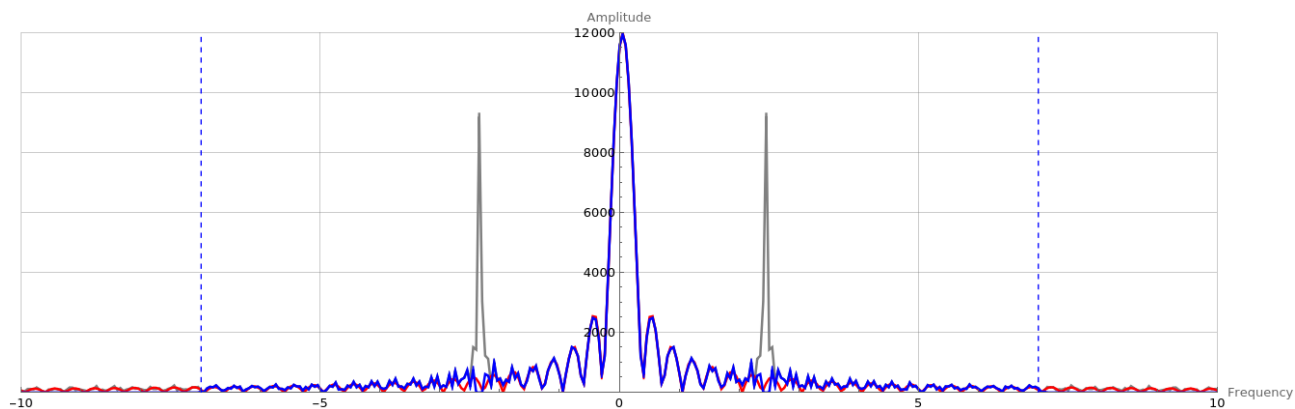


Рис. 27: Фурье-образы

Выбранные параметры: $\nu_0 = 7, c = 1, d = 15, \mathbf{b} = 1.5$

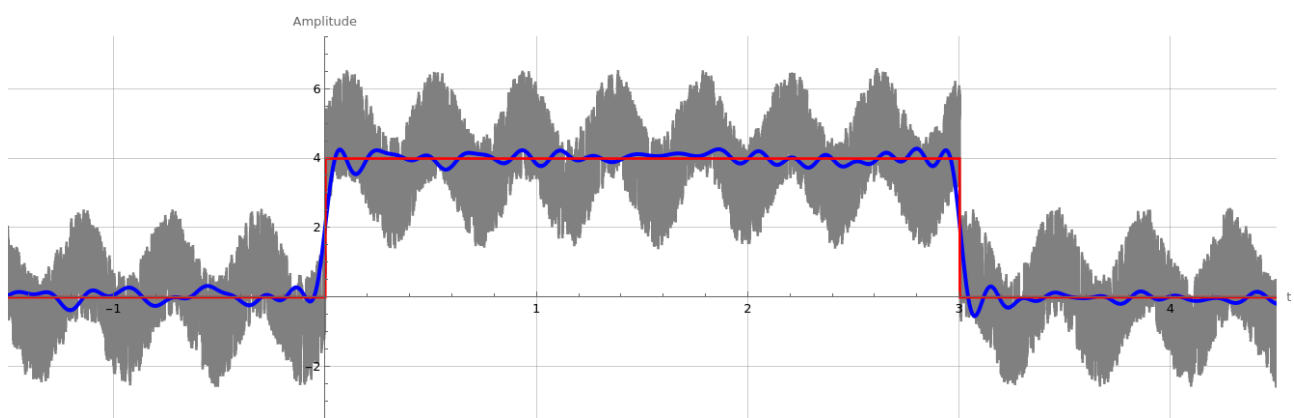


Рис. 28: Сигналы

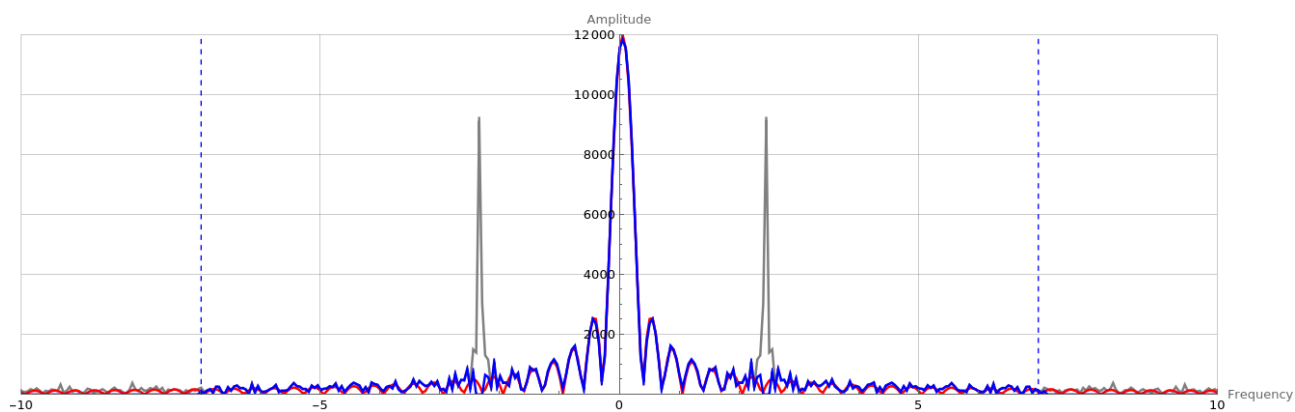


Рис. 29: Фурье-образы

- Влияние параметра d

Выбранные параметры: $\nu_0 = 7, c = 1, \mathbf{d} = 5, b = 0.5$

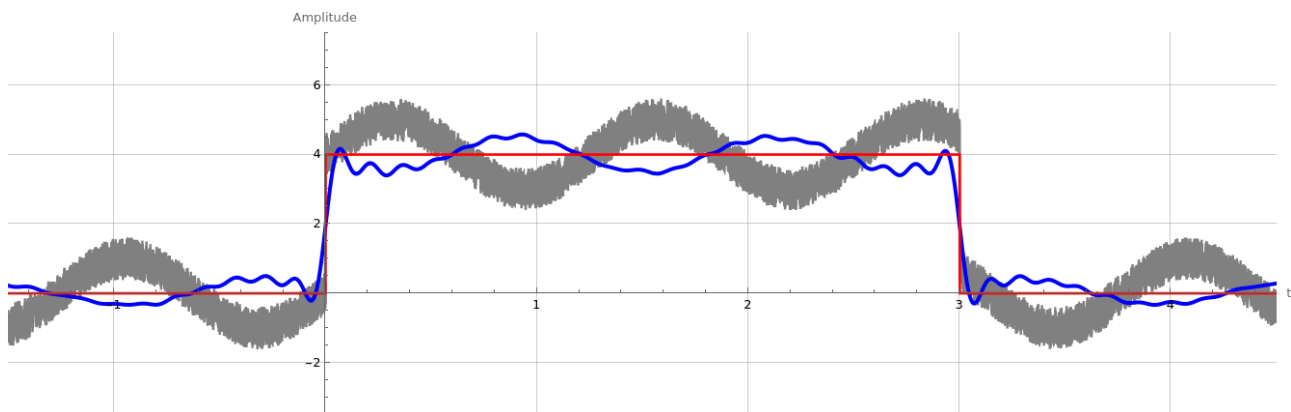


Рис. 30: Сигналы

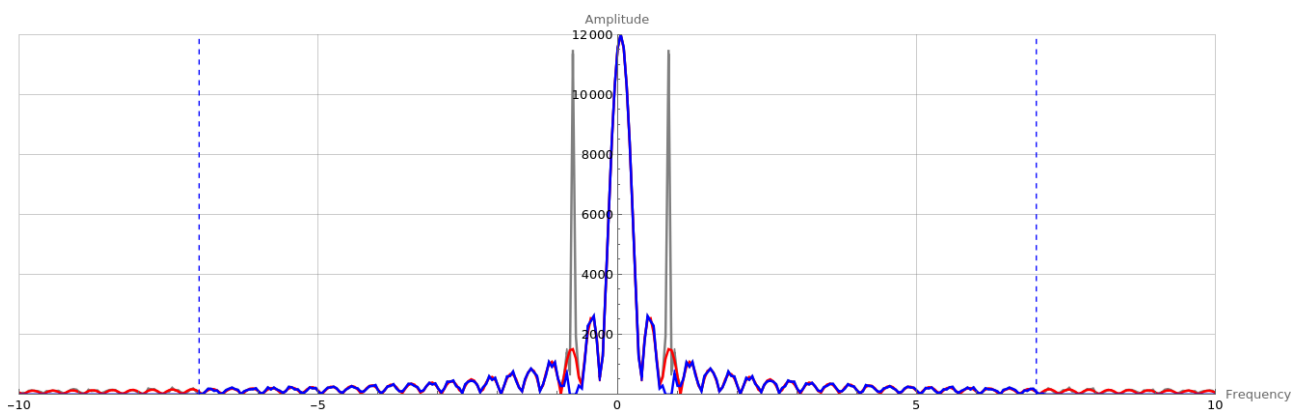


Рис. 31: Фурье-образы

Выбранные параметры: $\nu_0 = 7, c = 1, \mathbf{d} = \mathbf{20}, b = 0.5$

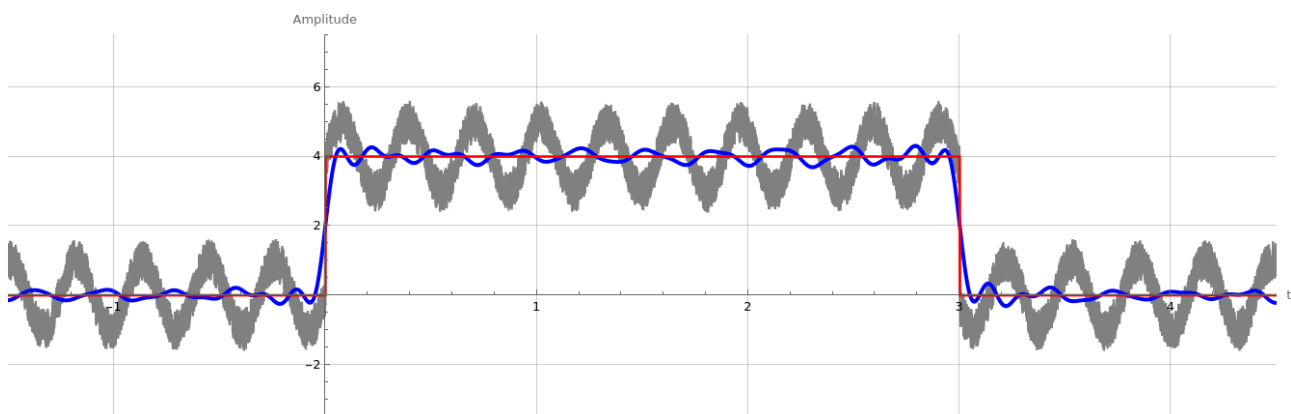


Рис. 32: Сигналы

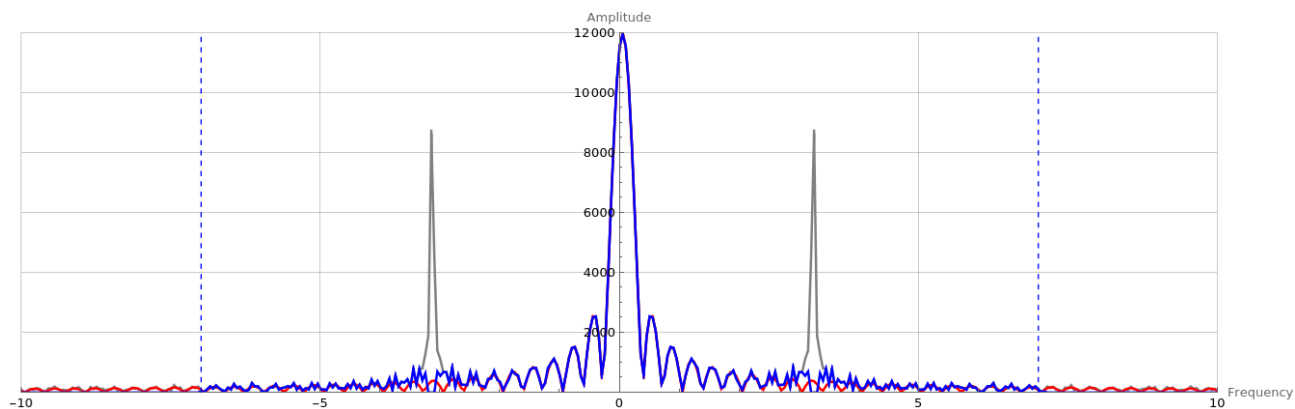


Рис. 33: Фурье-образы

Выбранные параметры: $\nu_0 = 7, c = 1, \mathbf{d} = \mathbf{35}, b = 0.5$

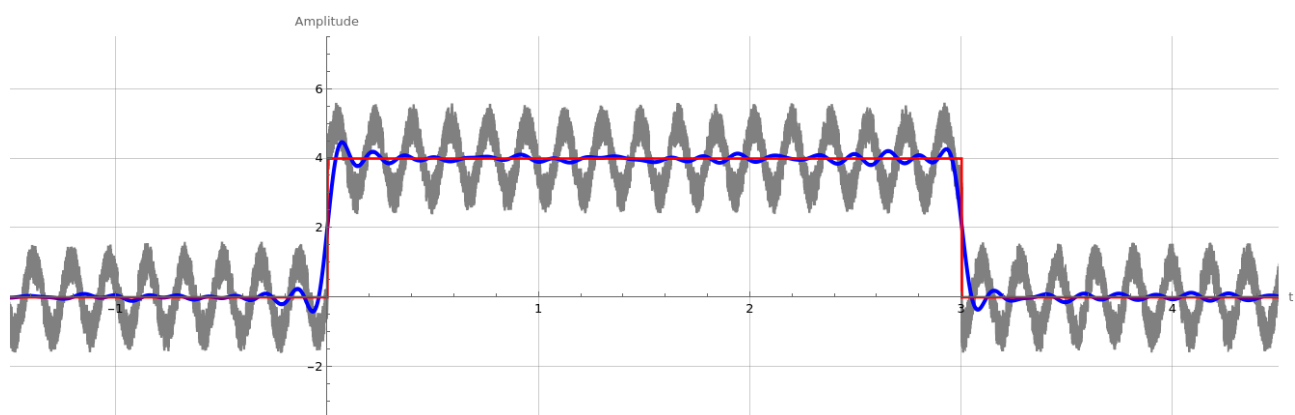


Рис. 34: Сигналы

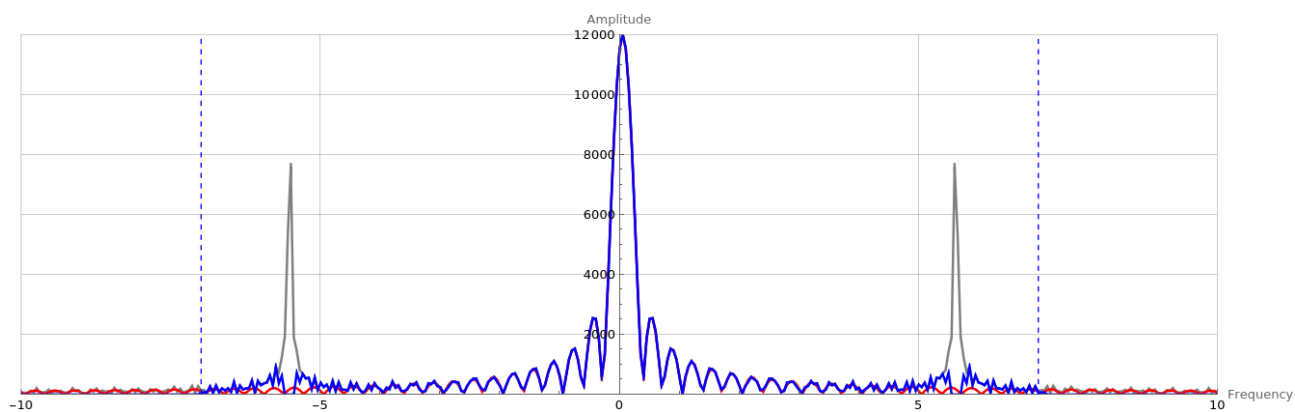


Рис. 35: Фурье-образы

- Влияние параметра c

Выбранные параметры: $\nu_0 = 7, \mathbf{c} = \mathbf{0.5}, d = 15, b = 0.5$

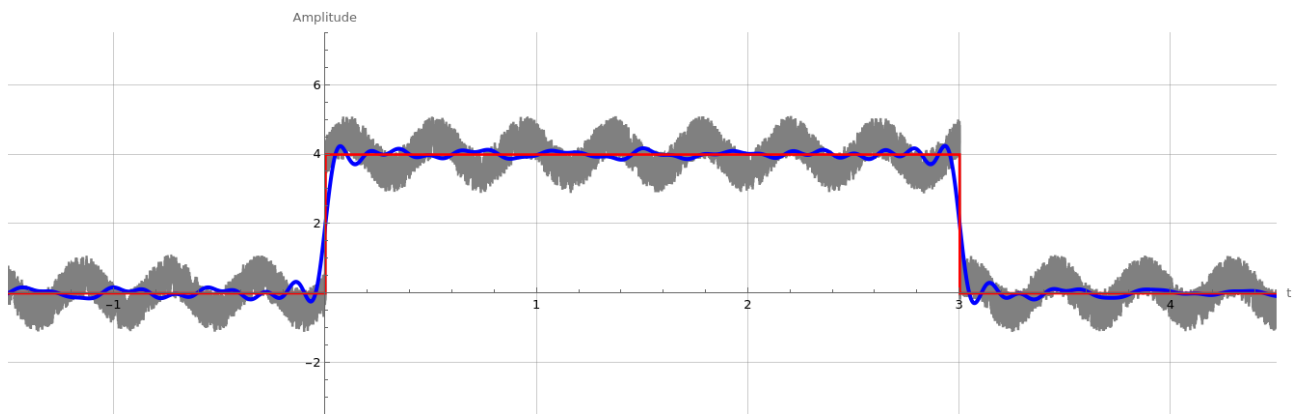


Рис. 36: Сигналы

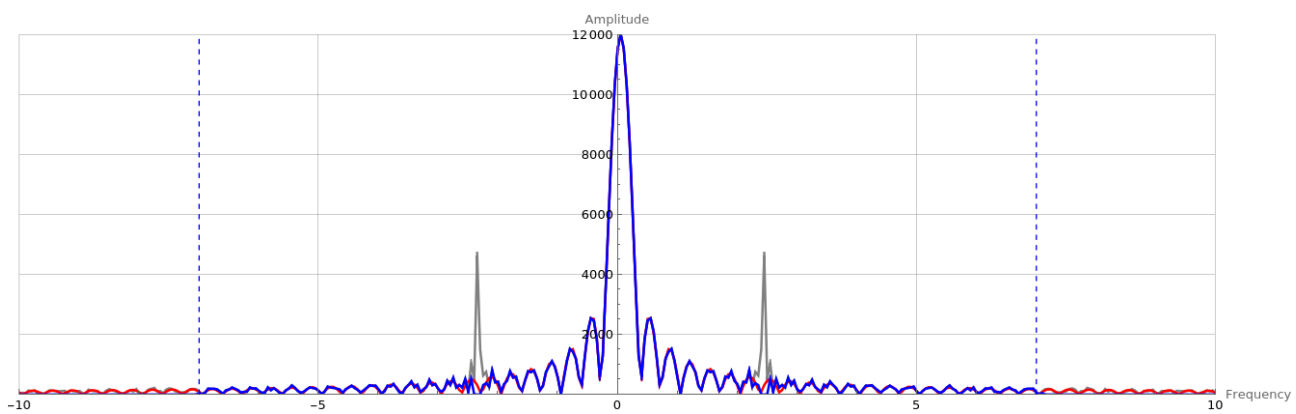


Рис. 37: Фурье-образы

Выбранные параметры: $\nu_0 = 7$, $c = 1.5$, $d = 15$, $b = 0.5$

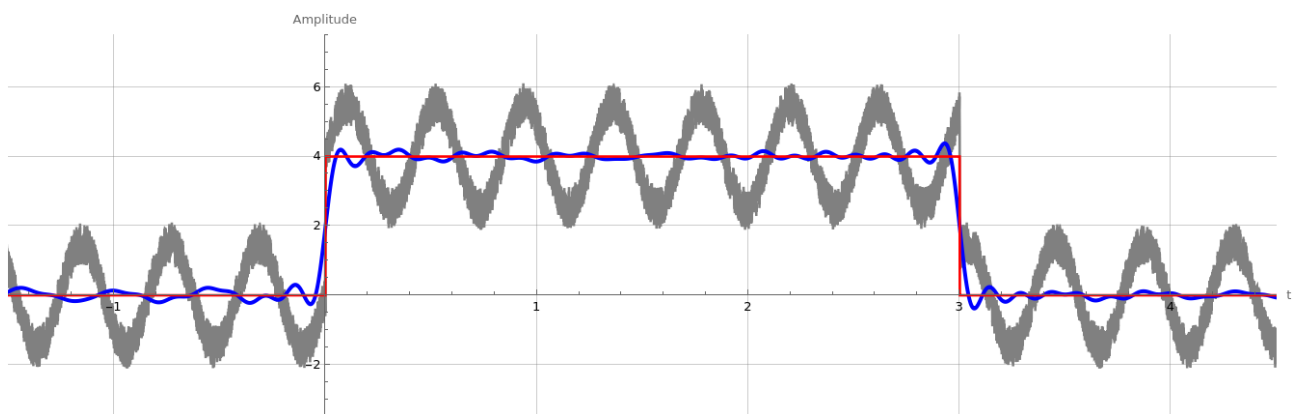


Рис. 38: Сигналы

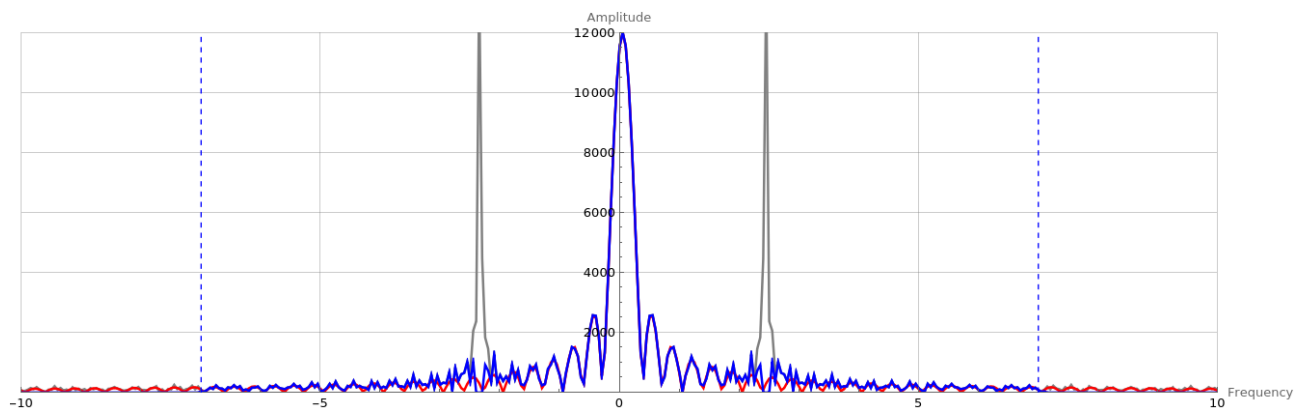


Рис. 39: Фурье-образы

Выбранные параметры: $\nu_0 = 7$, $c = 3$, $d = 15$, $b = 0.5$

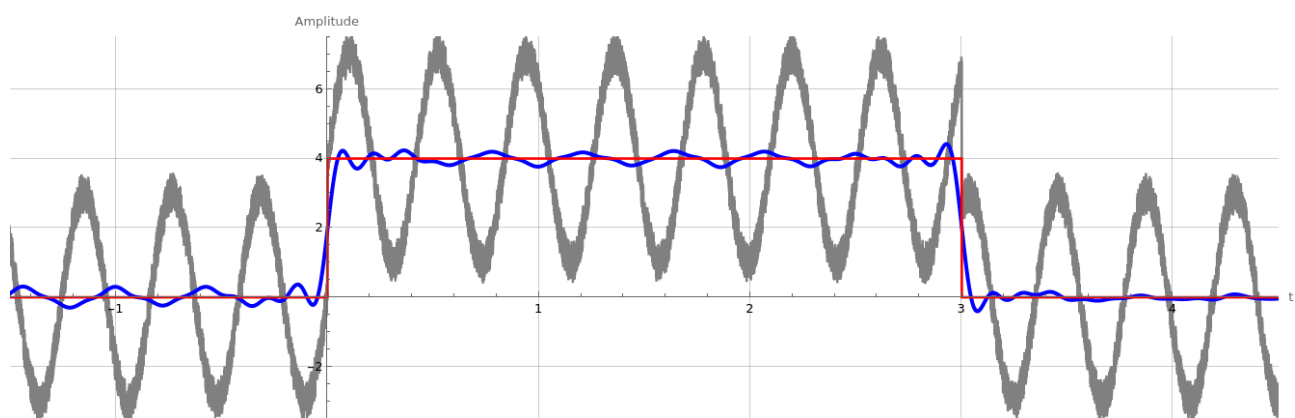


Рис. 40: Сигналы

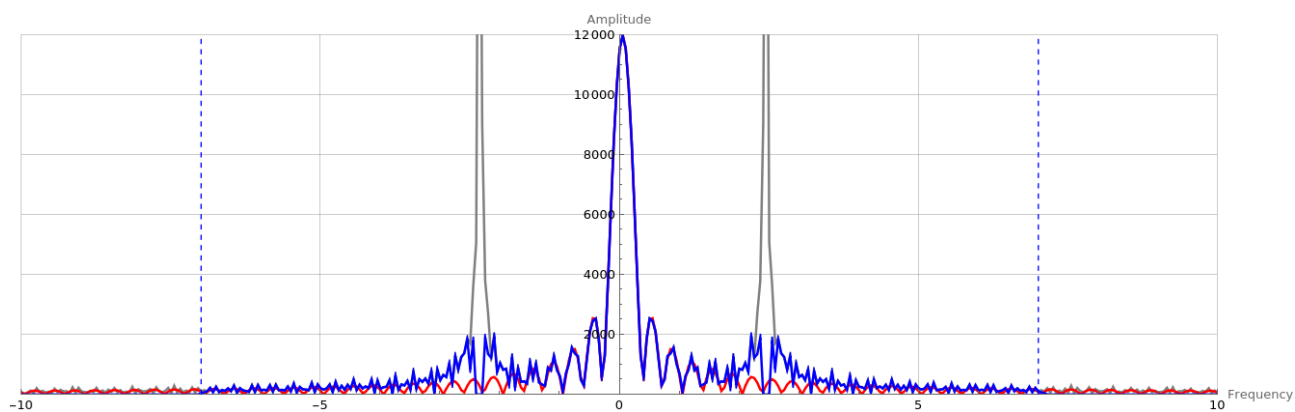


Рис. 41: Фурье-образы

- Влияние параметра ν_0

Выбранные параметры: $\nu_0 = 3$, $c = 1$, $d = 15$, $b = 0.5$

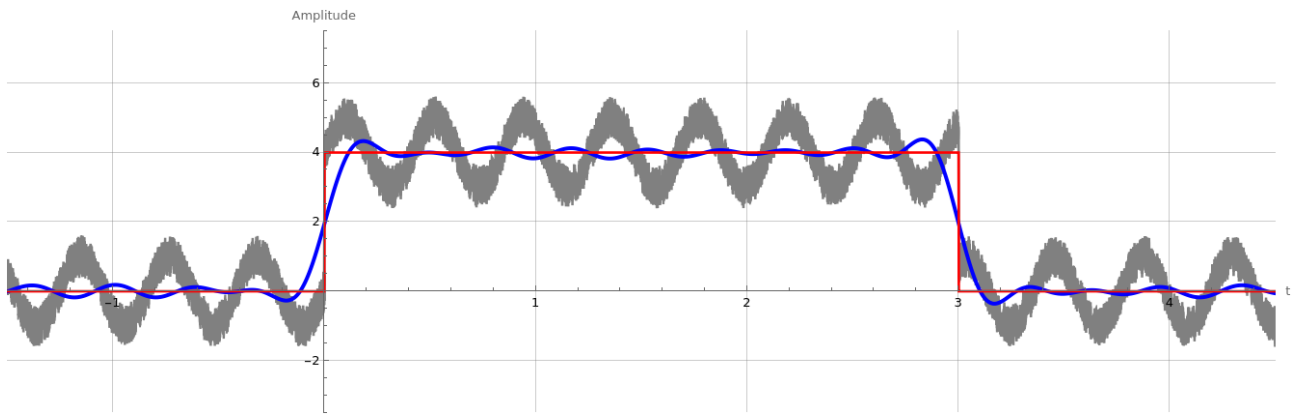


Рис. 42: Сигналы

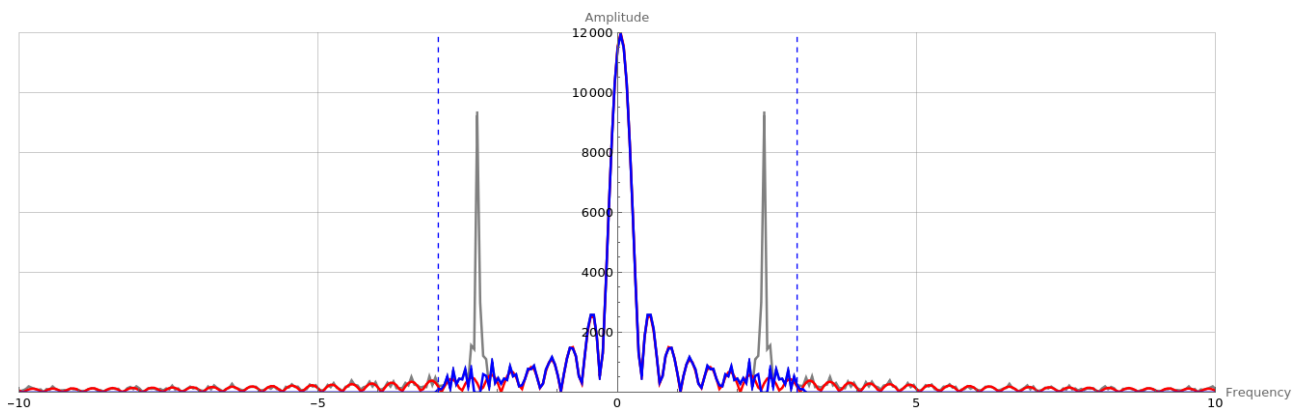


Рис. 43: Фурье-образы

Выбранные параметры: $\nu_0 = 5$, $c = 1$, $d = 15$, $b = 0.5$

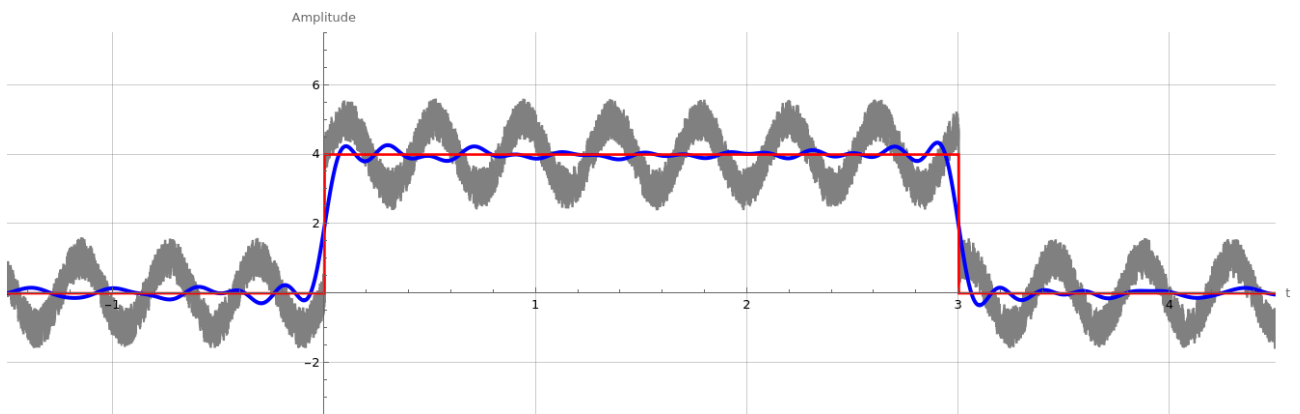


Рис. 44: Сигналы

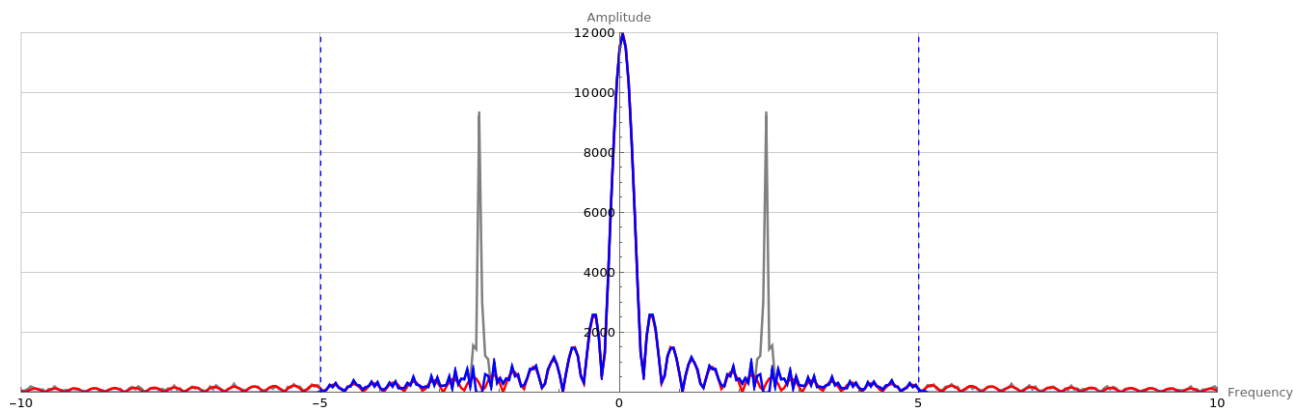


Рис. 45: Фурье-образы

Выбранные параметры: $\nu_0 = 10$, $c = 1$, $d = 15$, $b = 0.5$

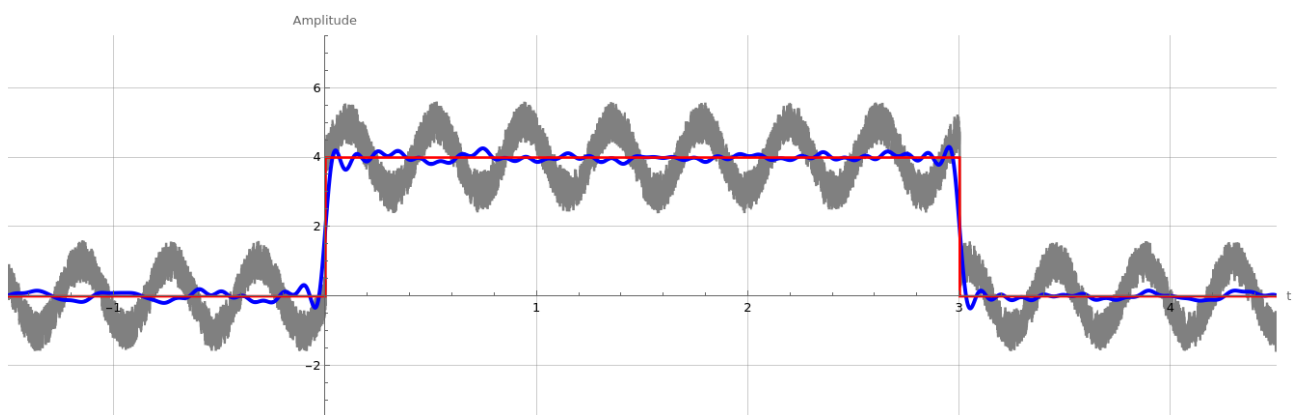


Рис. 46: Сигналы

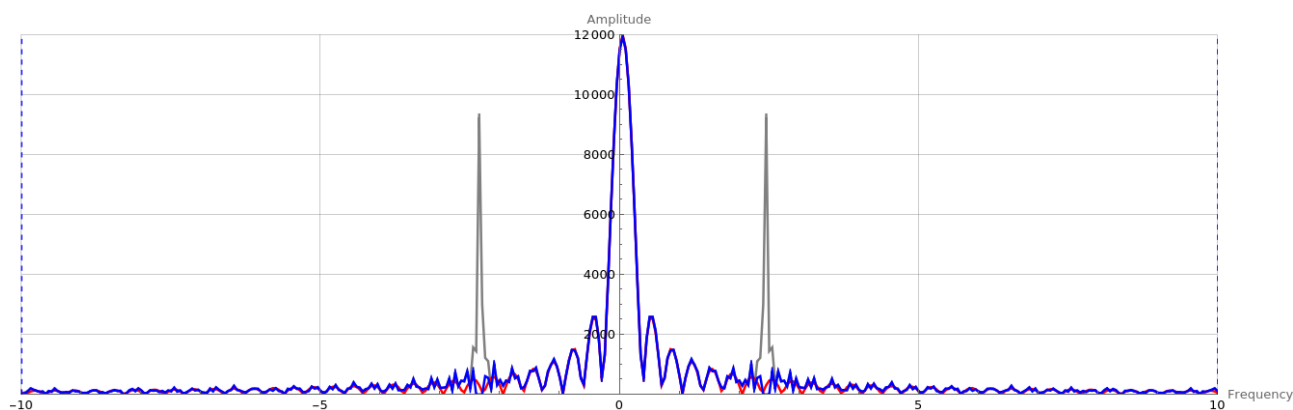


Рис. 47: Фурье-образы

1.3.3 Вывод

Параметр b определяет интенсивность шума. Чем больше параметр, тем больше шума будет у сигнала. При $b = 0$ не требуется убирать помехи шума, нужно убирать только гармонику.

Параметр d отвечает за частотные показатели гармонического шума.

c - характеризует мощность ("высоту") гармонических помех. Но качество фильтрации практически не изменяется при фиксированных параметрах.

Параметр ν_0 контролирует сколько высокочастотных составляющих останется в фильтрованном сигнале.

1.4 Убираем низкие частоты?

1.4.1 Предподготовка

Для начала в этом задании я выберу такие функции:

$$g(t) = \begin{cases} 4, & t \in [0, 3], \\ 0, & t \notin [0, 3], \end{cases}$$

$$u(t) = g(t) + 0.5\xi(t) + \sin(15t),$$

где $\xi(t) \sim U[-1, 1]$ — равномерное распределение на интервале $[-1, 1]$.

И пропущу функцию $u(t)$ через фильтр, который обнуляет фурье-образ в окрестности точки $\nu = 0$. Окрестность регулируется параметром $\Delta\nu$.

Алгоритм фильтрации:

- Вычисляю спектр сигнала: `Ushifted`
- Создаю маску, обнуляющую узкую полосу вокруг частоты $\nu_0 = 0$: `zeroAroundZeroMask`
- Применяю фильтр: умножаю маску на спектр `Ufiltered = Ushifted*zeroAroundZeroMask` и делаю обратное преобразование Фурье

1.4.2 Графики

Изобразю график для выбранных ранее параметров. Буду менять окрестность точки $\nu - \Delta\nu = \{0, 0.5, 1\}$ и оценю результаты.

- Значения: $b = 0.5$, $c = 1$, $d = 15$ и $\Delta\nu = 0$

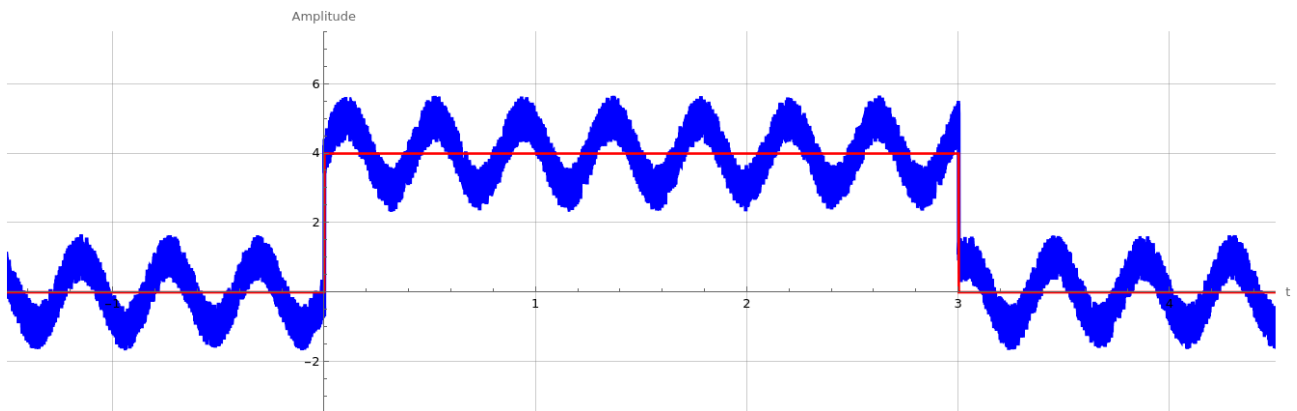


Рис. 48: Сигналы

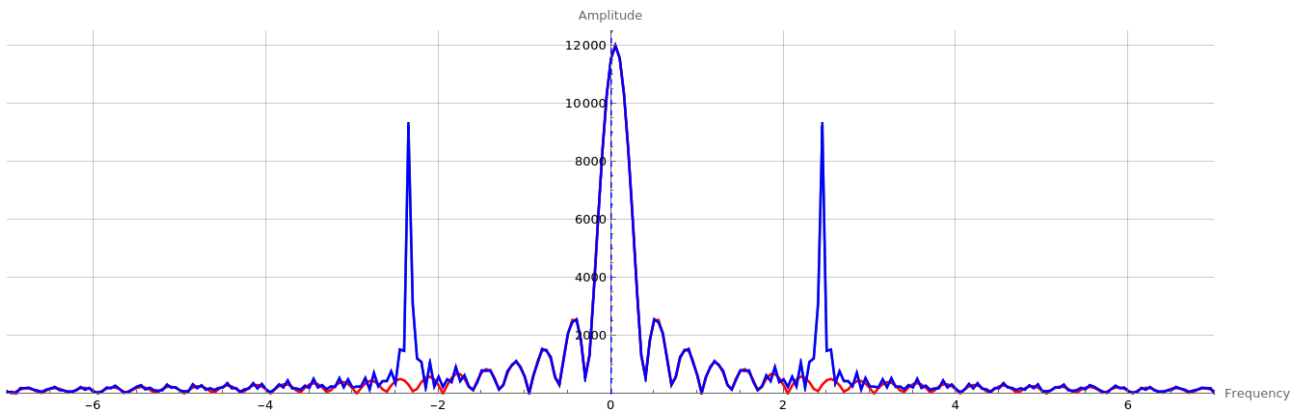


Рис. 49: Фурье-образы

- Значения: $b = 0.5$, $c = 1$, $d = 15$ и $\Delta\nu = 0.5$

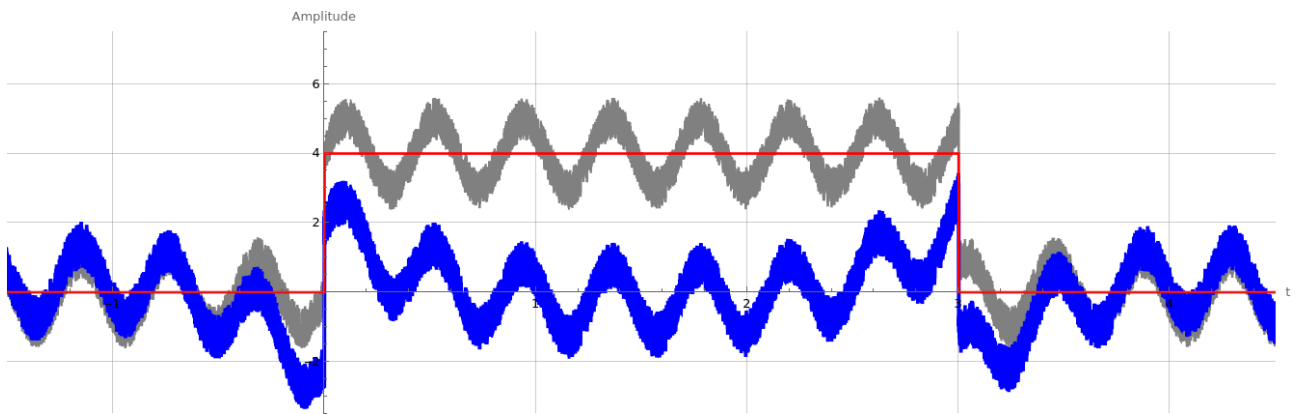


Рис. 50: Сигналы

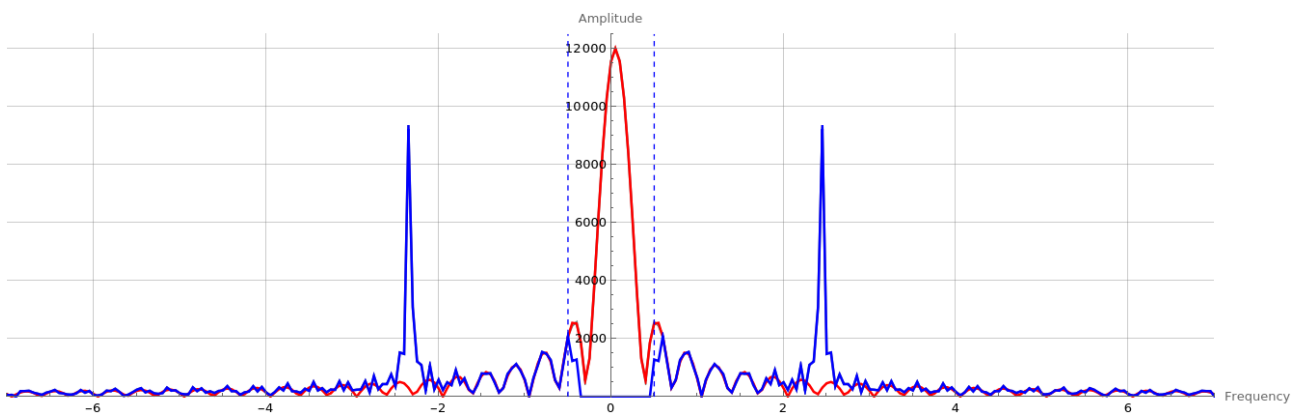


Рис. 51: Фурье-образы

- Значения: $b = 0.5$, $c = 1$, $d = 15$ и $\Delta\nu = 1$

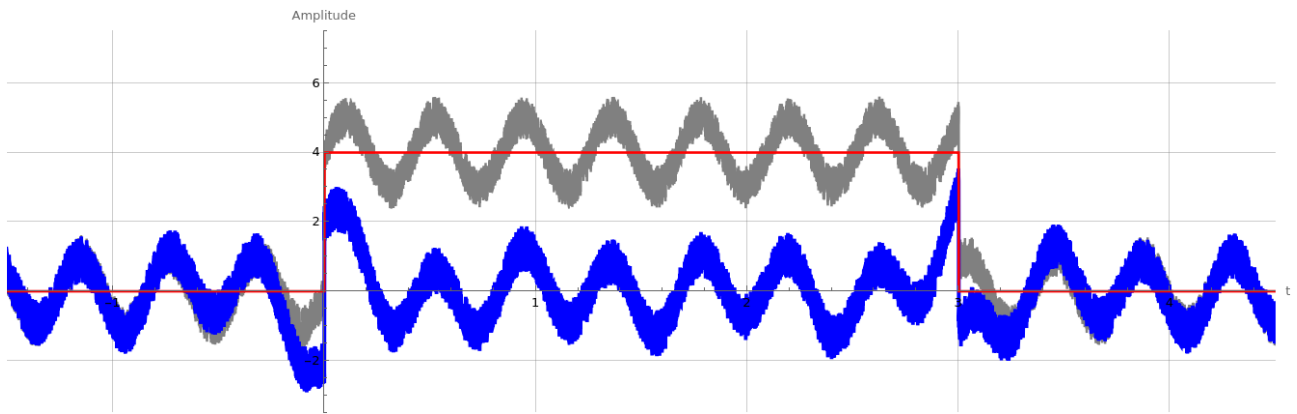


Рис. 52: Сигналы

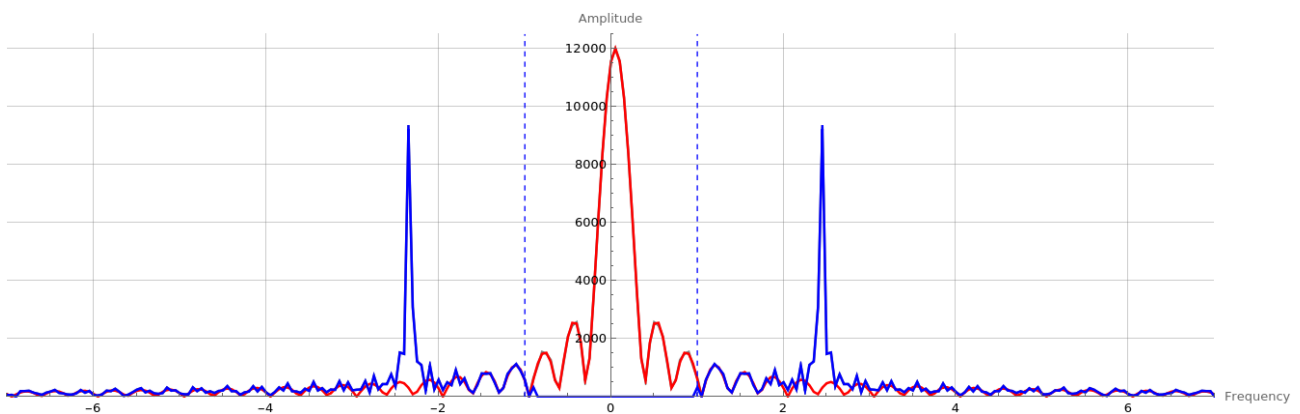


Рис. 53: Фурье-образы

1.4.3 Выводы

- На графиках видно, что при разных значениях частот среза изменяется степень фильтрации и форма сигнала.
- Фильтр успешно подавляет низкочастотные компоненты сигнала, что видно по изменению формы отфильтрованного сигнала и его Фурье-образа. Низкочастотные компоненты вблизи ν практически отсутствуют после фильтрации.
- Однако данный фильтр не является идеальным решением. Подавление низких частот в сигнале может быть не всегда целесообразным, так как низкочастотные компоненты часто содержат полезную информацию. Кроме того, шум в высокочастотной области сигнала сохраняется, что может ухудшить качество обработки.

Таким образом, проведенный анализ подтверждает, что фильтр низких частот эффективно справляется с задачей подавления низкочастотных компонент сигнала, но его использование может быть ограничено из-за сохранения шума и возможного искажения полезной информации.

2 Task. Фильтрация звука

2.1 Краткое условие

Дан аудиофайл `MUNA.wav`, содержащий речь и шум. Необходимо отфильтровать сигнал так, чтобы осталась только речевая часть.

Ожидаемые результаты:

- Построены графики исходного и фильтрованного сигналов во времени;
- Построены графики модулей Фурье-образов этих сигналов;
- Описан выбранный диапазон фильтрации и причины выбора.

2.2 График исходного сигнала

Для начала построю график исходного сигнала. В питоне для этого к записи нужно применить `sf.read("MUHA.wav")`. И построить график зависимости амплитуды от времени. Посмотрим на получившийся график:

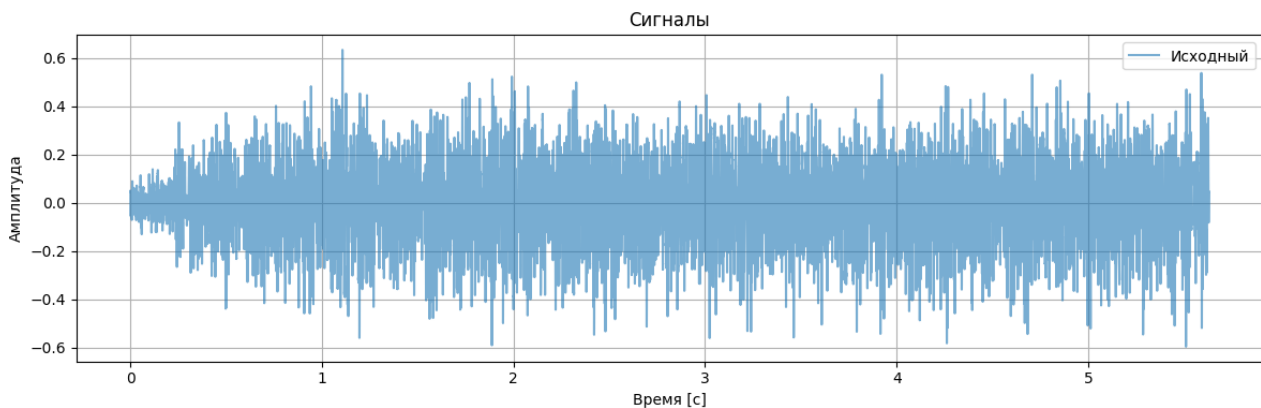


Рис. 54: График сигнала

Послушав запись, я выяснила, что шум существует на низкой частоте. Я отфильтровала низкие частоты и послушала получившуюся запись. Оказалось, что в записи есть еще немного высокого писка. Поэтому нужно отфильтровать и его. В качестве фильтра для этого задания я выбрала полосовой, который фильтрует сигнал в диапазоне. Размеры диапазона были подобраны самостоятельно.

Применю фильтр к сигналу. Оставлю частоты $\in [500, 3000]$ Гц:

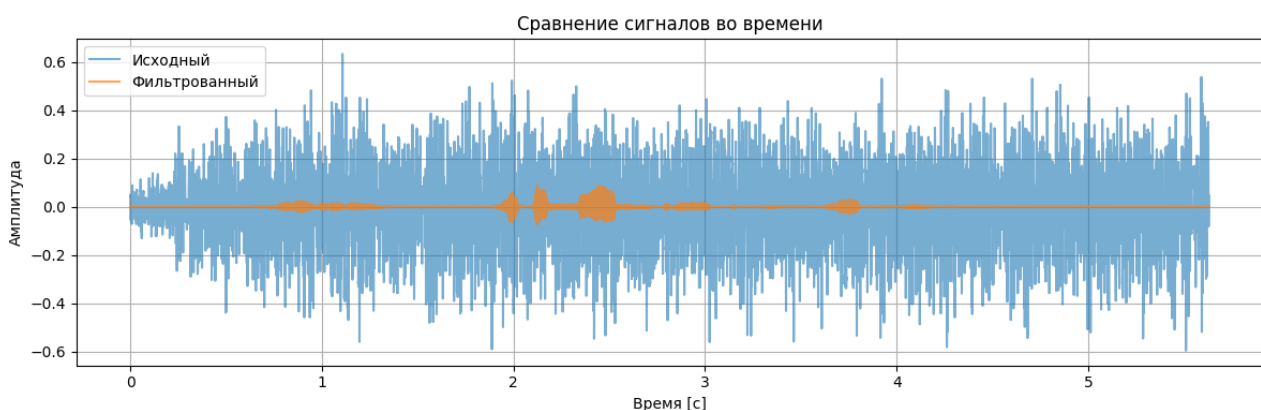


Рис. 55: График сигналов

Отфильтрованный сигнал можно послушать вот тут : [filtreted](#).

Можно заметить, что фильтр отлично сработал, много шума было удалено. Теперь посмотрим на графики модулей Фурье-образов исходного и фильтрованного сигналов:



Рис. 56: График спектра

На графике спектра видно, что после фильтрации осталась только часть сигнала в диапазоне от 500 до 3000 Гц. Низкие и высокие частоты были удалены — именно там находился основной шум.

Благодаря этому фильтр хорошо выделил голос. Основная энергия осталась в нужной полосе, а лишние шумы — исчезли. Это позволило сделать звук намного чище, при этом речь сохранилась.

3 Выводы

В этой работе я научилась фильтровать сигналы в частотной области. Сначала я работала с простыми функциями и шумами, потом — с реальной аудиозаписью. Я подобрала подходящий полосовой фильтр, который хорошо убрал лишние частоты и оставил только голос. Благодаря этому сигнал стал чище, а результат — лучше слышен. Всё это помогло мне лучше понять, как работает частотная фильтрация на практике. [GitHub](#).

4 Приложение

```

1 a = 4;
2 t1 = 0;
3 t2 = 3;
4 c = 0;
5 d = 5;
6 b = 0.5;
7
8 T = 20;
9 dt = 0.001;
10 tValues = Range[-T/2, T/2, dt];
11 n = Length[tValues];
12
13 vValues = Table[-1/(2 dt) + (k - 1)*1/T, {k, n}];
14
15 gVals = Table[If[t1 <= t <= t2, a, 0], {t, tValues}];
16 xi = RandomReal[{-1, 1}, n];

```

```

17 uVals = gVals + b*xi + c*Sin[d*tValues];
18
19 ListLinePlot[{Transpose[{tValues, uVals}],
20   Transpose[{tValues, gVals}]}],
21   AspectRatio -> 0.2,
22   PlotLegends -> Placed[{"Original", "Noisy"}, Right],
23   PlotStyle -> {{Gray, Thickness[0.002]}, {Red, Thickness[0.004]}},
24   AxesLabel -> {"t", "Amplitude"},
25   ImageSize -> 1000,
26   PlotRange -> {{-1, 4}, {-1, 5}}
27 ]

```

Листинг 1: Инициализация исходного и зашумленного сигналов

```

1 v = 5;
2 U = Fourier[uVals, FourierParameters -> {1, -1}];
3 Ushifted = RotateLeft[U, Floor[n/2]];
4
5 filterMask = Table[If[Abs[f] <= v, 1, 0], {f, vValues}];
6 Ufiltered = Ushifted*filterMask;
7 UfilteredBack = RotateRight[Ufiltered, Floor[n/2]];
8 uFilteredVals =
9   InverseFourier[UfilteredBack, FourierParameters -> {1, -1}];
10
11 ListLinePlot[{Transpose[{tValues, uVals}],
12   Transpose[{tValues, Re[uFilteredVals]}],
13   Transpose[{tValues, gVals}]}],
14   PlotStyle -> {{Gray, Thickness[0.002]}, {Blue,
15     Thickness[0.003]}, {Red, Thickness[0.002]}},
16   AxesLabel -> {"t", "Amplitude"},
17   PlotLegends -> Placed[{"Noisy", "Filtered", "Original"}, Right],
18   AspectRatio -> 0.3,
19   ImageSize -> 1000,
20   PlotRange -> {{-1.5, 4.5}, {-5, 9}},
21   GridLines -> Automatic
22 ]
23 (*****)
24 G = Fourier[gVals, FourierParameters -> {1, -1}];
25 Gshifted = RotateLeft[G, Floor[n/2]];
26
27 UfilteredSpec =
28   Fourier[Re[uFilteredVals], FourierParameters -> {1, -1}];
29 UfilteredShifted = RotateLeft[UfilteredSpec, Floor[n/2]];
30
31 widthN = 5000;
32
33 ListLinePlot[{Transpose[{vValues, Abs[Ushifted]}],
34   Transpose[{vValues, Abs[Gshifted]}],
35   Transpose[{vValues, Abs[UfilteredShifted]}]}],
36   PlotStyle -> {{Gray, Thickness[0.002]}, {Red,
37     Thickness[0.002]}, {Blue, Thickness[0.002]}},
38   AxesLabel -> {"Frequency", "Amplitude"},
39   PlotLegends -> Placed[{"Noisy", "Original", "Filtered"}, Right],
40   AspectRatio -> 0.3,

```

```

41 ImageSize -> 1000,
42 PlotRange -> {{-10, 10}, {-10, widthN}},
43 Epilog -> {
44   Dashed, Blue,
45   Line[{{v, 0}, {v, widthN}}],
46   Line[{{-v, 0}, {-v, widthN}}]
47 },
48 GridLines -> Automatic
49 ]

```

Листинг 2: Фильтрация низких частот. Отрисовка сигналов и фурье-образов

```

1 U = Fourier[uVals, FourierParameters -> {1, -1}];
2 Ushifted = RotateLeft[U, Floor[n/2]];
3
4 v = 10;
5 harmFreq = d/(2 Pi);
6 notchWidth = 0.13;
7
8 lowPassMask = Table[If[Abs[f] <= v, 1, 0], {f, vValues}];
9 notchMask =
10 Table[If[
11 Abs[f - harmFreq] < notchWidth || Abs[f + harmFreq] < notchWidth,
12 0, 1], {f, vValues}];
13 combinedMask = lowPassMask*notchMask;
14
15 Ufiltered = Ushifted*combinedMask;
16 UfilteredBack = RotateRight[Ufiltered, Floor[n/2]];
17 uFilteredVals =
18 InverseFourier[UfilteredBack, FourierParameters -> {1, -1}];
19
20 ListLinePlot[{Transpose[{tValues, uVals}],
21   Transpose[{tValues, Re[uFilteredVals]}],
22   Transpose[{tValues, gVals}]}],
23 PlotStyle -> {{Gray, Thickness[0.002]}, {Blue,
24   Thickness[0.003]}, {Red, Thickness[0.002]}},
25 AxesLabel -> {"t", "Amplitude"},
26 PlotLegends -> Placed[{"Noisy", "Filtered", "Original"}, Right],
27 AspectRatio -> 0.3,
28 ImageSize -> 1000,
29 PlotRange -> {{-1.5, 4.5}, {-3.5, 7.5}},
30 GridLines -> Automatic
31 ]
32 (*****)
33 G = Fourier[gVals, FourierParameters -> {1, -1}];
34 Gshifted = RotateLeft[G, Floor[n/2]];
35
36 UfilteredSpec =
37 Fourier[Re[uFilteredVals], FourierParameters -> {1, -1}];
38 UfilteredShifted = RotateLeft[UfilteredSpec, Floor[n/2]];
39
40 plotWigth = 12000;
41 ListLinePlot[{Transpose[{vValues, Abs[Ushifted]}],
42   Transpose[{vValues, Abs[Gshifted]}],

```

```

43   Transpose[{vValues, Abs[UfilteredShifted]}]],
44   PlotStyle -> {{Gray, Thickness[0.002]}, {Red,
45     Thickness[0.002]}, {Blue, Thickness[0.002]}},
46   AxesLabel -> {"Frequency", "Amplitude"},
47   PlotLegends -> Placed[{"Noisy", "Original", "Filtered"}, Right],
48   AspectRatio -> 0.3,
49   ImageSize -> 1000,
50   PlotRange -> {{-10, 10}, {-10, plotWigth}},
51   Epilog -> {
52     Dashed, Blue,
53     Line[{{v, 0}, {v, plotWigth}}],
54     Line[{{-v, 0}, {-v, plotWigth}}]
55   }, GridLines -> Automatic
56 ]

```

Листинг 3: Фильтрация специфических частот. Отрисовка сигналов и фурье-образов

```

1  U = Fourier[uVals, FourierParameters -> {1, -1}];
2  Ushifted = RotateLeft[U, Floor[n/2]];
3
4  v = 0;
5  notchWidthAroundZero = 1;
6
7  zeroAroundZeroMask =
8  Table[If[Abs[f - v] < notchWidthAroundZero, 0, 1], {f, vValues}];
9
10 Ufiltered = Ushifted*zeroAroundZeroMask;
11 UfilteredBack = RotateRight[Ufiltered, Floor[n/2]];
12 uFilteredVals =
13 InverseFourier[UfilteredBack, FourierParameters -> {1, -1}];
14
15 ListLinePlot[{Transpose[{tValues, uVals}],
16   Transpose[{tValues, Re[uFilteredVals]}],
17   Transpose[{tValues, gVals}]}],
18 PlotStyle -> {{Gray, Thickness[0.002]}, {Blue,
19   Thickness[0.003]}, {Red, Thickness[0.002]}},
20 AxesLabel -> {"t", "Amplitude"},
21 PlotLegends -> Placed[{"Noisy", "Filtered", "Original"}, Right],
22 AspectRatio -> 0.3,
23 ImageSize -> 1000,
24 PlotRange -> {{-1.5, 4.5}, {-3.5, 7.5}},
25 GridLines -> Automatic
26 ]
27 (*****
28 G = Fourier[gVals, FourierParameters -> {1, -1}];
29 Gshifted = RotateLeft[G, Floor[n/2]];
30
31 UfilteredSpec =
32 Fourier[Re[uFilteredVals], FourierParameters -> {1, -1}];
33 UfilteredShifted = RotateLeft[UfilteredSpec, Floor[n/2]];
34
35 plotN = 12500;
36
37 ListLinePlot[{Transpose[{vValues, Abs[Ushifted]}],

```

```

38 Transpose[{vValues, Abs[Gshifted]}],
39 Transpose[{vValues, Abs[UfilteredShifted]}]},
40 PlotStyle -> {{Gray, Thickness[0.002]}, {Red,
41   Thickness[0.002]}, {Blue, Thickness[0.002]}},
42 AxesLabel -> {"Frequency", "Amplitude"},
43 PlotLegends -> Placed[{"Noisy", "Original", "Filtered"}, Right],
44 AspectRatio -> 0.3,
45 ImageSize -> 1000,
46 PlotRange -> {{-7, 7}, {-20, plotN}},
47 Epilog -> {
48   Dashed, Blue,
49   Line[{notchWidthAroundZero, 0}, {notchWidthAroundZero, plotN}],
50   Line[{-notchWidthAroundZero, 0}, {-notchWidthAroundZero, plotN}]
51 },
52 GridLines -> Automatic
53 ]

```

Листинг 4: Фильтрация высоких частот. Отрисовка сигналов и фурье-образов

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import soundfile as sf
4 from scipy.signal import butter, filtfilt
5 from scipy.fft import fft, fftfreq
6
7 data, samplerate = sf.read("MUHA.wav")
8
9 def bandpass_filter(data, lowcut, highcut, fs, order=5):
10 nyq = 0.5 * fs
11 low = lowcut / nyq
12 high = highcut / nyq
13 b, a = butter(order, [low, high], btype='band')
14 return filtfilt(b, a, data)
15
16
17 lowcut = 500
18 highcut = 3000
19 filtered_data = bandpass_filter(data, lowcut, highcut, samplerate)
20
21 sf.write("MUHA_filtered.wav", filtered_data, samplerate)
22
23 T = len(data) / samplerate
24 time = np.linspace(0, T, len(data))
25
26 plt.figure(figsize=(12, 4))
27 plt.plot(time, data, label="Original", alpha=0.6)
28 plt.plot(time, filtered_data, label="Filtered", alpha=0.7)
29 plt.title("Signals in time")
30 plt.xlabel("Time (s)")
31 plt.ylabel("Amplitude")
32 plt.legend()
33 plt.grid()
34 plt.tight_layout()
35 plt.show()

```

```

36
37 N = len(data)
38 xf = fftfreq(N, 1 / samplerate)[:N // 2]
39 yf_orig = fft(data)
40 yf_filt = fft(filtered_data)
41
42 amp_orig = 2.0 / N * np.abs(yf_orig[:N // 2])
43 amp_filt = 2.0 / N * np.abs(yf_filt[:N // 2])
44
45 amp_orig /= np.max(amp_orig)
46 amp_filt /= np.max(amp_filt)
47
48 plt.figure(figsize=(12, 5))
49 plt.plot(xf, amp_orig, label="Original")
50 plt.plot(xf, amp_filt, label="Filtered")
51 plt.title("Spectrums")
52 plt.xlabel("GZ")
53 plt.ylabel("Norm Amplitude")
54 plt.xlim(0, 2000)
55 plt.grid(True)
56 plt.legend()
57 plt.tight_layout()
58 plt.show()

```

Листинг 5: Задание 2. Фильтрация сигнала